

구조적 문서화 체계 CWEB

(버전 3.64 — 2002년 2월)

Donald E. Knuth와 Silvio Levy

남수진 옮김

T_EX은 미국 수학회(American Mathematical Society)의 상표다.

Acrobat Reader는 Adobe Systems Incorporated의 상표다.

이 설명서의 인쇄본에 대한 저작권은 1994년 Addison-Wesley Publishing Company, Inc.에 있다. 모든 권리를 보유한다.

전자본에 대한 저작권은 1987, 1990, 1993, 2000년 Silvio Levy와 Donald E. Knuth에게 있다.

The printed form of this manual is copyright © 1994 by Addison-Wesley Publishing Company, Inc. All rights reserved.

The electronic form is copyright © 1987, 1990, 1993, 2000 by Silvio Levy and Donald E. Knuth.

Permission is granted to make and distribute verbatim copies of the electronic form of this document provided that the electronic copyright notice and this permission notice are preserved on all copies.

Permission is granted to copy and distribute modified versions of the electronic form of this document under the conditions for verbatim copying, provided that the entire resulting derived work is distributed under the terms of a permission notice identical to this one.

Individuals may make copies of the documentation from the electronic files for their own personal use.

이 문서는 위 허가 고지에 따라 원문을 한국어로 옮긴 것이다. 옮기면서 생긴 잘못은 모두 옮긴이의 책임이다.

인터넷 페이지 <http://www-cs-faculty.stanford.edu/~knuth/cweb.html>에 CWEB과 관련 주제에 대한 최신 소식이 있다.

구조적 문서화 체계 CWEB

Donald E. Knuth와 Silvio Levy

이 문서는 Don Knuth의 WEB 체계를 Silvio Levy가 C에 맞게 고친 판을 설명한다. 1987년에 만들어진 뒤로 CWEB은 Knuth와 Levy 두 사람의 손을 거쳐 여러 모로 다듬어지고 나아졌다. 이제 그 진화는 거의 끝에 이르렀다고 우리는 믿는다. 그래도 버그 보고와 제안, 의견은 언제나 반갑고, Levy (levy@math.berkeley.edu)에게 보내면 된다.

Knuth의 각서 “The WEB System of Structured Documentation”에 익숙한 독자라면 이 글을 빠르게 훑어도 좋다. CWEB과 WEB은 철학이 같고 문법도 (본질적으로) 같기 때문이다. 어떤 면에서 CWEB은 WEB을 단순하게 만든 것이다. 이를테면 C와 그 전처리기가 이미 매크로와 문자열을 다뤄 주므로 CWEB에는 WEB의 매크로 정의와 문자열 처리 기능이 필요 없다. 마찬가지로 8진수와 16진수 상수를 @'77과 @"3f로 적던 WEB의 관례는 C의 관례인 077과 0x3f로 바뀌었다. WEB의 다른 기능은 모두 그대로 남았고, 새 기능이 더해졌다.

CWEB을 만드는 데 제안과 비판을 보태 준 모든 이에게 고마움을 전한다. 특히 코드를 보태 준 Steve Avery, Nelson Beebe, Hans-Hermann Bode, Klaus Guntermann, Norman Ramsey, Joachim Schnitter, Saroj Mahapatra, 그리고 이 설명서를 여러 모로 낮게 해 준 제안을 준 Cameron Smith에게 깊이 감사한다. Ramsey는 자신의 SPIDER 체계로 또 다른 언어의 사용자들도 문학적 프로그래밍을 할 수 있게 했다 [*Communications of the ACM* **32** (1989), 1051–1055를 보라]. Knuth의 책 *Literate Programming* (1992)에는 관련된 초기 연구의 방대한 참고문헌이 실려 있다.

Bode와 Schnitter, Mahapatra는 CWEB이 C++에서도 돌아가도록 고쳤다. 그러니 아래 글에서 C라고 한 것은 원한다면 C++로 읽어도 된다.

들어가며

CWEB의 바탕에 깔린 철학은 이렇다. 자기 프로그램에 가능한 한 가장 좋은 문서를 붙이고 싶은 프로그래머에게는 두 가지가 동시에 필요하다. 조판을 위한 $\text{\TeX}$ 같은 언어와, 프로그래밍을 위한 C 같은 언어가 그것이다. 어느 한 쪽만으로는 가장 좋은 문서를 만들 수 없다. 그러나 둘을 알맞게 엮으면 각각을 따로 쓸 때보다 훨씬 쓸모 있는 체계를 얻는다.

소프트웨어 프로그램의 구조는 서로 얽힌 여러 조각으로 이루어진 “거미줄(web)” 이라고 생각할 수 있다. 그런 프로그램을 문서로 만들려면 거미줄의 각 부분이 무엇이고 이웃과 어떻게 이어지는지를 설명해야 한다. $\text{\TeX}$ 이 주는 조판 도구는 각 부분의 국소적인 구조를 눈에 보이게 만들어 설명할 기회를 주고, C가 주는 프로그래밍 도구는 알고리즘을 형식에 맞게 모호함 없이 적을 수 있게 해 준다. 이 둘을 엮으면, 복잡한 소프트웨어의 구조를 최대한 잘 파악하게 해 주면서도 문서와 맞아떨어지는 실제 소프트웨어로 기계적으로 옮길 수 있는, 그런 프로그래밍 방식을 펼칠 수 있다.

CWEB 체계는 CWEAVE와 CTANGLE이라는 두 프로그램으로 이루어진다. CWEB 프로그램을 쓸 때 사용자는 C 코드와 문서를 한 파일에 함께 둔다. 이 파일을 CWEB 파일이라 하고 보통 something.w처럼 이름 붙인다. ‘cweave something’ 명령은 출력 파일 something.tex을 만든다. 이 파일을 $\text{\TeX}$ 에 넣으면 something.w의 “예쁘게 적은” 판이 나오는데, 쪽 배치와 들여쓰기, 이탤릭, 볼드, 수학 기호 같은 조판의 세부를 제대로 처리해 준다. 조판된 출력에는 자동으로 모은 방대한 상호 참조 색인도 들어 있다. 마찬가지로 ‘ctangle something’ 명령을 돌리면 C 파일 something.c가 나오고, 이것을 컴파일하면 실행 코드가 된다.

CWEB은 문서화 도구일 뿐 아니라, 프로그램 본문의 조각들을 마음대로 순서를 바꿔 놓을 수 있게 하여 C 언어를 확장한다. 그래서 큰 체계도 작은 절들과 그들 사이의 국소적인 관계만으로 온전히 이해할 수 있다. CTANGLE이라는 이름은 주어진 거미줄에서 절들을 뽑아 CWEB 구조에서 C가 요구하는 순서로 옮겨 놓기(tangle) 때문에 붙었다. CWEB으로 프로그래밍하는 이점은 알고리즘을 “엮이지 않은” 꼴로, 절마다 따로 설명하며 적을 수 있다는 것이다. CWEAVE라는 이름은 주어진 거미줄에서 각 절에 담긴 $\text{\TeX}$ 부분과 C 부분을 서로 엮은 뒤, 그 천 전체를 구조화된 문서로 짜내기(weave) 때문에 붙었다. (알아채셨는지? 놀랍지 않은가.) 독일어로 “weave”가 “webe”이고 그에 대응하는 라틴어 명령형이 “texe”라는 사실과 무슨 깊은 인연이 있는지도 모르겠다!

CWEB 사용자는 C 프로그래밍 언어에 어지간히 익숙해야 한다. $\text{\TeX}$ 도 조금은 알면 좋지만, 사실 그건 CWEB을

쓰면서 익혀도 된다. 평범한 글이라면 그 언어를 거의 몰라도 $\text{\TeX}$ 으로 조판할 수 있기 때문이다. C와 $\text{\TeX}$ 둘 다에 익숙한 사람이라면 CWEB의 명령을 익히는 데 드는 수고는 작다.

한눈에 보기

CWEB 파일에는 두 갈래의 재료가 들어간다. $\text{\TeX}$ 텍스트와 C 텍스트다. CWEB으로 글을 쓰는 프로그래머는 만들어지고 있는 문서와 C 프로그램을 동시에 생각해야 한다. 다시 말해 CWEAVE와 CTANGLE이 CWEB 파일에 저마다 어떤 일을 할지가 몸에 배어 있어야 한다. $\text{\TeX}$ 텍스트는 CWEAVE가 사실상 그대로 베껴 내고 CTANGLE은 통째로 지운다. $\text{\TeX}$ 텍스트는 “순수한 문서”인 셈이다. 반면 C 텍스트는 CWEAVE가 조판하고 CTANGLE이 규칙에 따라 이리저리 옮겨 놓는다. 그 규칙은 차차 밝혀질 것이다. 지금은 두 갈래의 텍스트가 있다는 것만 마음에 새겨 두면 된다. CWEB 프로그램을 쓰는 일은 $\text{\TeX}$ 문서를 쓰는 일과 비슷하되, $\text{\TeX}$ 의 수평 모드와 수직 모드, 수학 모드에 “C 모드”가 하나 더 붙은 셈이다.

CWEB 파일은 어지간히 그 자체로 온전한 절(section)이라는 단위를 쌓아 만든다. 절은 저마다 세 부분으로 이루어진다.

- $\text{\TeX}$ 파트. 그 절에서 무슨 일이 벌어지는지를 설명하는 글이 담긴다.
- 중간 파트. 매번 다 풀어 쓰면 알아보기 어려운 C 구문을 줄여 부르는 매크로 정의가 담긴다. CTANGLE은 이것을 전처리기 매크로 정의로 바꾼다.
- C 파트. CTANGLE이 만들어 낼 프로그램의 한 조각이 담긴다. 이 C 코드는 열두어 줄쯤이 가장 좋다. 그래야 한 덩이로 쉽게 이해되고 구조도 한눈에 들어온다.

절의 세 부분은 반드시 이 순서로 나와야 한다. 즉 $\text{\TeX}$ 주석이 먼저, 그다음이 중간 파트, 마지막이 C 코드다. 어느 부분이든 비어 있어도 된다.

절은 ‘@_’나 ‘@*’ 기호로 시작한다. 여기서 ‘_’는 빈칸을 뜻한다. 절은 다음 절이 시작하는 곳(즉 다음 ‘@_’나 ‘@*’)에서 끝나거나, 그보다 파일의 끝이 먼저 오면 거기서 끝난다. CWEB 파일에는 어떤 절에도 속하지 않는 재료가 있을 수도 있다. 바로 첫 절보다 앞에 오는 글이다. 그런 글은 “limbo에 있다”고 하는데, CTANGLE은 무시하고 CWEAVE는 사실상 그대로 베껴 낸다. 그러니 그 구실은 $\text{\TeX}$ 출력에 넣고 싶은 조판 지시를 더해 주는 것이다. 실제로 CWEB 파일은 특별한 글꼴을 불러오고 특별한 매크로를 정의하고 쪽 크기를 바꾸고 표제지를 만드는 $\text{\TeX}$ 코드를 limbo에 두고 시작하는 것이 보통이다.

절에는 1부터 차례로 번호가 매겨진다. 이 번호는 CWEAVE가 내놓는 $\text{\TeX}$ 문서의 각 절 첫머리에 나타나고, CTANGLE이 내놓는 C 프로그램에서는 그 절이 만들어 낸 코드의 앞뒤에 대괄호 주석으로 나타난다.

절 이름

다행히도 CWEB으로 글을 쓸 때 그 번호를 직접 입에 올릴 일은 없다. 새 절이 시작하는 곳마다 ‘@_’나 ‘@*’라고만 적으면 번호는 CWEAVE와 CTANGLE이 알아서 붙여 준다. 쓰는 사람에게 절은 번호가 아니라 이름을 갖는다. 이름은 ‘@<’ 다음에 $\text{\TeX}$ 텍스트를 쓰고 ‘@>’로 닫아 적는다. CWEAVE는 절 이름을 내놓을 때 ‘@<’와 ‘@>’를 꺾쇠로 바꾸고 그 안에 절 번호를 작은 글자로 끼워 넣는다. 그래서 CWEAVE의 출력을 읽을 때 다른 절에서 언급한 절을 찾기가 쉽다.

설명을 위해서라면 절 이름은 그 절의 내용을 잘 나타내야 한다. 즉 그 절이 대표하는 추상을 이름이 대신해야 한다. 그러면 그 절을 다른 한두 절에 “꽃아” 넣으면서 속사정의 자잘한 부분은 감출 수 있다. 그러니 절 이름은 뜻이 제대로 전해질 만큼 길어야 한다.

안타깝게도 그렇게 긴 이름을 몇 번이고 치는 것은 고된 일이고, 긴 이름을 두 번 똑같이 적어 CWEAVE와 CTANGLE이 이름과 절을 맞춰 볼 수 있게 하기도 어렵다. 이 사정을 덜어 주려고 CWEAVE와 CTANGLE은 절 이름을 줄여 쓸 수 있게 해 준다. 온전한 이름이 CWEB 파일 어딘가에 나오기만 하면 된다. ‘@< α ...@>’라고만 쓰면 되는데, 여기서 α 는 파일에 나오는 절 이름 가운데 꼭 하나의 앞머리가 되는 문자열이다. 이를테면 ‘Clear’로 시작하는 절 이름이 달리 없다면 ‘@<Clear the arrays@>’는 ‘@<Clear...@>’로 줄여 쓸 수 있다. 다른 곳에서는 ‘@<Clear t...@>’처럼 써도 된다.

그 밖에는 절 이름이 글자 하나하나까지 맞아야 한다. 다만 이어지는 흰 칸(빈칸, 탭, 줄바꿈, 폼피드)은 빈칸 하나와 같게 치고, 이름의 앞뒤에 있는 그런 빈칸은 지워진다. 그러므로 ‘@< Clear the arrays @>’도 앞선 보기의 이름과 맞는다. 줄임표 뒤의 빈칸도 무시되지만 앞의 것은 그렇지 않다. 그래서 ‘@<Clear t ...@>’는 ‘@<Clear the arrays@>’와 맞지 않는다.

CTANGLE이 하는 일

절이 ‘@_’나 ‘@*’로 시작한다고는 했지만, 그것이 어떻게 TeX 파트와 중간 파트, C 파트로 갈리는지는 말하지 않았다. 중간 파트는 그 절에서 ‘@d’나 ‘@f’가 처음 나오는 곳에서 시작하고, C 파트는 ‘@c’나 ‘@<절 이름@>’이 처음 나오는 곳에서 시작한다. 뒤엎것을 쓰면 그 절 이름이 뒤따르는 C 텍스트를 대신한다고 말하는 셈이다. 반대로 C 파트가 절 이름 대신 ‘@c’로 시작하면 그 절은 이름 없는 절이라고 한다.

‘@<절 이름@>’이라는 구성은 절의 C 파트에 몇 번이고 나올 수 있다. 두 번째부터 나오는 것은 이름 있는 절을 “정의”하는 게 아니라 “쓴다”는 뜻이다. 달리 말해 다른 데서 정의되었을 그 이름 있는 절의 C 코드를 C 프로그램의 바로 이 자리에 끼워 넣으라는 것이다. 사실 CTANGLE의 큰 뜻은 이름 있는 절과 이름 없는 절들로 C 프로그램 하나를 만들어 내는 데 있다. 그 일이 정확히 어떻게 이루어지는지는 이렇다. 먼저 ‘@d’로 지시한 매크로 정의를 모두 C 전처리기 매크로 정의로 바꿔 맨 앞에 베껴 놓는다. 그다음 이름 없는 절들의 C 파트를 차례대로 베껴 내린다. 이것이 프로그램 본문의 1차 근사다. (이름 없는 절이 적어도 하나는 있어야 한다. 없으면 프로그램도 없다.) 그다음 1차 근사에 나오는 모든 절 이름을 그에 대응하는 절의 C 파트로 바꾸고, 절 이름이 하나도 남지 않을 때까지 이 치환을 이어 간다. 주석은 모두 지워진다. C 프로그램은 오로지 C 컴파일러의 눈만을 위한 것이기 때문이다.

같은 이름이 여러 절에 붙어 있으면 그 이름에 대응하는 C 텍스트는 그 절들의 C 파트를 모두 이어 붙여 얻는다. 이 기능은 이를테면 ‘Global variables’라는 이름의 절에서 쓸모가 있다. 전역 변수를 그 변수가 등장하는 절에서 바로 선언할 수 있기 때문이다. 여러 절이 같은 이름을 가지면 CWEAVE는 첫 절의 번호를 그 이름의 번호로 삼고, 그 절 아래에 ‘아무 절도 보라’는 알림을 넣어 같은 이름을 가진 다른 절들의 번호를 일러 준다. 절에 대응하는 C 텍스트는 보통 CWEB 파일의 등호 자리에 항등 기호가 오도록 CWEAVE가 조판한다. 즉 출력은 ‘< 절 이름 > ≡ C 텍스트’가 된다. 다만 같은 이름의 절이 두 번째 이후로 나올 때는 이 ‘≡’ 기호가 ‘+≡’로 바뀌어, 뒤따르는 C 텍스트가 다른 절의 C 텍스트에 덧붙는 것임을 알려 준다.

CTANGLE은 절에 들고 날 때마다 C 출력 파일에 전처리기 #line 명령을 끼워 넣는다. 그래서 컴파일러가 오류 메시지를 낼 때나 프로그램을 디버깅할 때 그 메시지가 C 파일이 아니라 CWEB 파일의 줄 번호를 가리킨다. 그러니 대개는 C 파일을 아예 잊고 지내도 된다.

CWEAVE가 하는 일

CWEAVE의 큰 뜻은 CWEB 파일에서 다음과 같이 .tex 파일을 만드는 것이다. .tex 파일의 첫 줄은 CWEB의 문서화 관례를 정의하는 매크로 파일을 읽어 들이라고 TeX에게 이른다. 그다음 줄들에는 첫 절 앞의 limbo에 있던 TeX 텍스트가 베껴진다. 그다음 각 절의 출력이 차례로 오는데 사이사이에 쪽 끝 표시가 끼기도 한다. 끝으로 CWEAVE는 각 C 식별자가 나오는 절 번호를 모은 상호 참조 색인을 만들고, 절 이름을 가나다순으로 늘어놓은 목록과, “별표 붙은” 절마다 쪽 번호와 절 번호를 보여 주는 목차도 만든다.

“별표 붙은” 절이 무엇이나고? ‘@_’ 대신 ‘@*’로 시작하는 절인데, 절 무리의 큰 마디가 새로 시작한다는 표시라는 점이 조금 특별하다. ‘@*’ 뒤에는 그 무리의 제목을 쓰고 마침표를 찍어야 한다. 그런 절은 TeX 출력에서 언제나 새 쪽에서 시작하고, 그 무리의 제목은 다음 별표 절이 나올 때까지 이어지는 모든 쪽의 running head로 나온다. 제목은 목차에도 나오고, 그 절의 첫머리에는 볼드로 나온다. 조심할 것: 그런 제목에는 TeX 제어열을 쓰지 말라. cwebmac 매크로가 그것을 제대로 다룬다고 확신할 때만 쓰라. 까닭은 이렇다. 이 제목은 running head로 나올 때 대문자로 바뀌고, 절 첫머리에 나올 때 볼드로 바뀌며, TeX이 일하는 동안 임시로 쓰는 목차 파일에도 적힌다. 그러니 쓰려는 제어열은 이 세 경우 모두에서 뜻이 통해야 한다.

CWEAVE가 각 절에 대해 내놓는 TeX 출력은 이렇게 이루어진다. 먼저 절 번호가 온다 (이를테면 123번 절의 첫머리에는 ‘\M123.’. 다만 별표 절의 첫머리에는 ‘\M’ 자리에 ‘\N’이 온다). 그다음 절의 TeX 파트가 오는데, 아래에 적은 것을 빼면 거의 그대로 베껴진다. 그다음 중간 파트와 C 파트가 오는데, 둘 다 비어 있지 않으면 사이가 조금 벌어지도록 조판된다. 중간 파트와 C 파트는 C 프로그램에 우스꽝스럽게 생긴 TeX 매크로를 잔뜩 끼워 넣어 얻는다. 이 매크로들은 글꼴과 알맞은 수식 간격 같은 조판의 세부는 물론 줄바꿈과 들여쓰기까지 도맡는다.

TeX 텍스트 속의 C 코드, 그리고 그 반대

TeX 텍스트를 치다 보면 C 코드의 변수를 비롯한 여러 양을 자주 가리키고 싶어질 것이고, 그 변수들이 글 속에 나올 때도 프로그램 속에 나올 때와 똑같은 모습으로 조판되기를 바랄 것이다. 그래서 CWEB 언어는 C 재료의 앞뒤에 ‘|’

표를 두면 $\text{\TeX}$ 텍스트 안에서도 C 편집의 효과를 얻을 수 있게 해 준다. 이를테면 이런 말을 하고 싶다고 하자.

pa 가 ‘`int *pa`’로 선언되어 있으면, 대입 `pa = &a[0]`은 pa 가 a 의 0번째 원소를 가리키게 한다.

CWEB 파일에서 이 $\text{\TeX}$ 텍스트는 이렇게 생겼다.

```
|pa|가 ‘|int *pa|’로 선언되어 있으면, 대입
|pa=&a[0]|은 |pa|가 |a|의 0번째 원소를
가리키게 한다.
```

그리고 CWEB은 이것을, 직접 치지 않아도 되어 다행이다 싶은 무엇으로 옮긴다.

```
\\{pa}가 ‘\&{int} ${}*\\{pa}$’로
선언되어 있으면, 대입 $\{pa}\K{\AND}\a[\T{0}]$은
\\{pa}가 \a의 0번째 원소를 가리키게 한다.
```

덧붙이자면, 이런 주석이 있을 때 CWEB이 만드는 상호 참조 색인에는 지금 절의 번호가 pa 의 색인 항목 가운데 하나로 들어간다. pa 가 이 절의 C 파트에는 나오지 않더라도 그렇다. 그러므로 색인은 프로그램 자체뿐 아니라 설명 주석에 나오는 식별자까지 아우른다. 이 기능의 고마움은 머지않아 알게 될 것이다. 다만 `int`와 a 는 색인에 오르지 않는다. CWEB은 예약어와 한 글자 식별자에 대해서는 색인 항목을 만들지 않기 때문이다. 그런 식별자는 너무나 흔해서 나오는 자리를 일일이 적어 봐야 부질없다고 본 것이다.

절이 $\text{\TeX}$ 텍스트로 시작해 C 텍스트로 끝난다고는 했지만, 그 경계가 뚜렷하지 않다는 것은 이미 보았다. ‘`!...`’로 감싸면 C 텍스트를 $\text{\TeX}$ 텍스트 안에 넣을 수 있으니 말이다. 거꾸로 $\text{\TeX}$ 텍스트도 C 텍스트 안에 자주 나타난다. 주석 안의 모든 것(즉 `/*`와 `*/` 사이, 또는 `//` 뒤)이 $\text{\TeX}$ 텍스트로 다루어지기 때문이다. 마찬가지로 절 이름의 본문도 $\text{\TeX}$ 텍스트이지만, `@<절 이름>`이라는 구성 전체는 C 텍스트에 놓이기를 기대한다. 그래서 아래 보기처럼 C 환경과 $\text{\TeX}$ 환경 사이를 자연스럽게 오가는 것이 보통이다.

```
if (x==0) @<Empty the |buffer| array>
... using the algorithm in |@<Empty the |buffer| array>|.
```

첫째 것은 어느 절의 C 파트에서 볼 만한 것으로, “Empty the *buffer* array”라는 이름의 절에 있는 코드가 그 자리에 끼워진다. 둘째 것은 그 절의 $\text{\TeX}$ 파트에서 볼 만한 것으로, 이름 있는 절을 정의하거나 쓰는 게 아니라 “인용”하는 것이다. (이때는 절 이름을 ‘`!...`’로 감쌌음에 주목하라.)

매크로

제어 코드 `@d` 뒤에

식별자 C 텍스트 또는 식별자(매개₁, ..., 매개_n) C 텍스트

가 오면 (둘째 경우 식별자와 괄호 사이에 빈칸이 없어야 한다) CTANGLE은 이것을 `#define`으로 시작하는 전처리기 명령으로 바꾸어, 앞서 설명한 대로 C 출력 파일의 맨 위에 찍는다.

‘`@d`’ 매크로 정의는 여러 줄에 걸쳐도 되고, 줄바꿈을 역슬래시로 막아 줄 필요도 없다. CTANGLE이 알아서 역슬래시를 넣어 주기 때문이다. 어떤 까닭에서든 C 파일의 특정한 자리에 `#define` 명령이 있어야 한다면, 그것을 CWEB 매크로가 아니라 C 코드로 다루면 된다. 다만 그때는 줄바꿈을 직접 막아야 한다.

문자열과 상수

여느 때처럼 ‘나 ’나 ” 쪽으로 둘러싼 문자열을 C 파일에 내보내고 싶다면 CWEB 파일에 그대로 치면 된다. 다만 ‘@’ 문자는 ‘@@’로 쳐야 한다 (그것은 제어 코드가 되는데, 문자열 안에 나올 수 있는 유일한 제어 코드다. 아래를 보라). 문자열은 시작한 줄에서 끝나야 한다. 줄 끝에 역슬래시가 있으면 이어 갈 수 있다.

$\text{\TeX}$ 과 C는 8진수와 16진수 상수를 가리키는 방식이 서로 다르다. $\text{\TeX}$ 은 기술적인 글쓰기를 겨냥하고 C은 컴퓨터의 처리를 겨냥하기 때문이다. $\text{\TeX}$ 에서는 상수 앞에 ‘나 ’나 ”를 붙여 8진수나 16진수로 만들고, C에서는 0이나 0x를 앞에 붙인다. CWEB에서는 각 관례를 제 영역에서 그대로 쓰게 하는 것이 이치에 맞아 보인다. 그래서 C 텍스트에서 40₈은 ‘040’이라고 치면 되는데, CTANGLE은 이것을 (컴파일러를 위해) C 파일에 그대로 베끼고

CWEAVE는 40으로 찍는다. 마찬가지로 CWEAVE는 16진 C 상수 '0x20'을 #20으로 찍는다. 8진 숫자에 이탤릭을, 16진 숫자에 타자기꼴을 쓰면 문서에서 그런 상수의 뜻이 더 또렷해진다. 그러니 일관되게, 절의 TeX 파트에서는 '1040i'이나 '10x20i'이라고 쳐야 한다.

제어 코드

CWEB의 제어 코드는 첫 글자가 '@'인 두 글자 조합이다. 몇몇 제어 코드의 뜻은 이미 보았다. 이제 좀 더 차근차근 늘어놓을 때다.

아래 목록에서 제어 코드 뒤 대괄호 안의 글자는 그 코드가 어떤 자리에서 쓰일 수 있는지를 알려 준다. *L*은 limbo에서 쓸 수 있다는 뜻이고, *T*(TeX), *M*(중간), *C*(C)는 절의 세 파트 각각에서 최상위 수준으로, 즉 '1...1'나 절 이름 같은 구성의 바깥에서 쓸 수 있다는 뜻이다. 화살표 →는 그 제어 코드가 CWEB 파일의 현재 파트를 끝내고 화살표 뒤 글자가 가리키는 파트를 연다는 뜻이다. 그러므로 @L 옆의 [LTM C → T]는 이 제어 코드가 limbo나 절의 세 파트 어디서든 나올 수 있고, 뒤따르는 절의 (비어 있을 수도 있는) TeX 파트를 연다는 뜻이다.

대괄호 안에는 두 가지 약자가 더 나올 수 있다. 글자 *r*은 제한된 문맥, 즉 C 주석과 절 이름, C 문자열, 그리고 (아래에서 정의할) 제어 텍스트 안의 재료를 뜻한다. 글자 *c*는 안쪽 C 문맥, 즉 '1...1' 안의 C 재료를 뜻한다 (주석 안의 '1...1'는 포함하되 다른 제한된 문맥 안의 것은 아니다). 대괄호 뒤의 별표 *는 이 제어 코드에서 짝이 되는 @>까지의 문맥이 제한된 문맥이라는 뜻이다.

글자가 들어가는 제어 코드는 대소문자를 가리지 않는다. 그러므로 @d와 @D는 같다. 아래에서는 소문자 쪽만 따로 적는다.

@ [LTM Crc] @를 둘 겹치면 '@' 한 글자를 뜻한다. 어디서나 쓸 수 있는 유일한 제어 코드다. CWEB 파일에 인터넷 전자우편 주소를 적을 때도 이 관례를 따라야 한다는 데 주의하라 (이를테면 levy@math.berkeley.edu).

다음은 절의 TeX 파트를 여는 코드들이다.

@ [LTM C → T] 새 (별표 없는) 절이 시작함을 나타낸다. @ 기호 뒤에 오는 탭이나 폼피드, 줄 끝 문자는 빈칸과 같다 (다른 대부분의 자리에서도 그렇다).

@* [LTM C → T] 새 별표 절, 즉 새 큰 무리를 여는 절이 시작함을 나타낸다. 새 무리의 제목을 @* 뒤에 쓰고 마침표를 찍어야 한다. 앞서 설명했듯이 그런 제목에는 아주 단순한 것이 아니라면 TeX 제어열을 쓰지 않는 게 좋다. CWEAVE와 CTANGLE은 @*를 읽으면 단말에 별표와 현재 절 번호를 찍어 사용자가 진행 상황을 가늠할 수 있게 한다. 맨 첫 절에는 별표를 붙여야 한다.

별표 절의 “깊이”는 @* 뒤에 *나 십진수를 쳐서 지정할 수 있다. 이것은 프로그램의 계층 구조에서 지금 절 무리가 갖는 상대적인 지위를 나타낸다. @**로 여는 프로그램의 최상위 부분은 목차에서 이름이 볼드로 조판되며, 깊이가 -1이라고 말한다. 그 밖의 깊이는 음이 아닌 수로, 목차 쪽에서 들여쓰는 양을 다스린다. 이 들여쓰기는 긴 프로그램의 구조를 또렷이 하는 데 도움이 된다. 따로 지정하지 않으면 깊이는 0으로 친다. 프로그램이 짧다면 깊이를 모두 0으로 두어도 무방하다. 별표 절은 깊이가 1보다 크지 않은 한 출력에서 언제나 새 쪽에서 시작한다.

각 절의 중간 파트는 몇 개든 상관없는 매크로 정의(@d로 시작)와 형식 정의(@f나 @s로 시작)가 순서에 매이지 않고 섞여 이루어진다.

@d [TM → M] 매크로 정의는 @d로 시작하고, 앞서 설명한 대로 식별자와 선택적인 매개변수, C 텍스트가 뒤따른다.

@f [TM → M] 형식 정의는 @f로 시작한다. 이것은 C 텍스트에 나오는 식별자를 CWEAVE가 특별하게 다루도록 한다. 형식 정의의 일반적인 꼴은 '@f l r'이고 뒤에 /*와 */로 감싼 주석이 올 수도 있다. 여기서 *l*과 *r*은 식별자이며, 그 뒤로 CWEAVE는 식별자 *l*을 지금 *r*을 다루는 방식대로 다룬다. 이 기능은 CWEB 프로그래머가 새 예약어를 지어내거나 C의 예약된 식별자 가운데 얼마를 예약에서 풀 수 있게 해 준다. 이를테면 흔한 낱말인 'error'와 'line'은 C 전처리기에서 특별한 뜻을 갖게 되어 CWEAVE도 이들을 특별하게 조판하도록 되어 있다. error나 line이라는 이름의 변수를 쓰고 싶다면 프로그램 어딘가에

```
@f error normal      @f line normal
```

이라고 적어야 한다.

r 이 특별한 식별자 ‘TeX’이면 식별자 l 은 TeX 제어열로 조판된다. 이를테면 CWEB 파일에 ‘@f foo TeX’ 이라고 적으면 CWEB은 식별자 foo 를 \foo로 내보낸다. 프로그래머는 TeX 수학 모드를 가정하고 \foo를 원하는 모양으로 정의하면 된다. (TeX 제어열을 만들 때 밑줄 문자는 x로, 달러 기호는 X로 바뀐다. 그러므로 $foo.bar$ 는 \fooxbar가 된다. 숫자를 포함한 다른 문자는 옮겨지지 않고 남으므로 TeX은 그것을 제어열의 일부가 아니라 매크로 매개변수로 여긴다. 이를테면

```
\def\x#1{x_{#1}} @f x1 TeX @f x2 TeX
```

이라고 하면 $x1$ 과 $x2$ 가 x_1, x_2 가 아니라 x_1, x_2 로 조판된다.)

r 이 특별한 식별자 ‘make_pair’이면 식별자 l 은 C++ 함수 템플릿으로 다루어진다. 이를테면 @f convert make_pair 뒤에는 <와 >가 작다. 크다 기호로 잘못 읽히는 일 없이 ‘convert<int>(2.5)’라고 쓸 수 있다.

CWEB은 typedef로 정의되는 식별자가 예약어가 되어야 한다는 것을 안다. 그러니 형식 정의가 필요한 일은 그리 잦지 않다.

@s $[TM \rightarrow M; L]$ @f와 같되, CWEB이 그 형식 정의를 출력에 보이지 않고, 선택적인 C 주석도 쓸 수 없다. 주로 @i 파일에서 쓴다.

다음은 절의 C 파트를 다스리는 코드들이다.

@c @p $[TM \rightarrow C]$ 이름 없는 절의 C 파트는 @c로 시작한다 (“program”의 @p로 시작해도 된다. 두 제어 코드는 같은 일을 한다). 이것은 3쪽에서 설명한 대로 CTANGLE이 뒤따르는 C 코드를 1차 프로그램 본문에 이어 붙이게 한다. CWEB은 TeX 출력에 ‘@c’를 찍지 않으니, TeX으로 찍은 CWEB 문서를 보고 CWEB 파일을 만들고 있다면 이름 없는 절의 알맞은 자리에 @c를 넣어야 한다는 것을 잊지 말아야 한다.

@< $[TM \rightarrow C; C; c]$ * 이 제어 코드는 절 이름(또는 앞서 말한 대로 헛갈리지 않는 앞머리)을 여는데, 그 이름은 TeX 텍스트로 이루어지며 짝이 되는 @>까지 이어진다. @<...@>라는 구성 전체는 개념상 하나의 C 요소다. 그 몸가짐은 문맥에 따라 다르다.

T 와 M 문맥에 나오는 @<는 뒤따르는 절 이름을 지금 절에 붙이고 그 절의 C 파트를 연다. 닫는 @> 뒤에는 =나 +=가 와야 한다.

C 문맥에서 @<는 이름 있는 절을 쓴다는 뜻이다. 3쪽에서 설명한 대로 CTANGLE이 그 절의 C 정의를 그 자리에 끼워 넣는다. 오류를 잡기 위한 장치로, 그런 절 이름 뒤에 =가 오면 CTANGLE과 CWEB이 불평한다. 십중팔구 새 절을 정의하려던 것이어서 앞에 @_가 있어야 하기 때문이다. <foo>이 정의가 아니라 쓰임인데도 정말로 <foo> = bar 라고 쓰고 싶다면 = 앞에 줄바꿈을 두라.

끝으로 안쪽 C 문맥에서는 (즉 절의 TeX 파트나 주석 안의 ‘|...|’ 안에서는) @<...@>가 이름 있는 절을 인용한다는 뜻이다. 그렇게 나온 것은 CTANGLE이 무시한다. 여기서도 절 이름을 C 요소로 여긴다는 데 주목하라. 그래서 |...|로 감싼다.

@($[TM \rightarrow C; C; c]$ * 절 이름은 @(로 시작할 수도 있다. 모든 것이 @<와 똑같이 돌아가되, @(foo@>라는 이름의 절에 담긴 C 코드를 CTANGLE이 foo 파일에 쓴다는 점만 다르다. 이렇게 하면 CWEB 파일 하나에서 여러 파일을 내보낼 수 있다. (@d 정의는 그런 파일에는 나가지 않고 으름 .c 파일에만 나간다.) 이 기능은 이를테면 지금 프로그램과 함께 링크될 다른 모듈을 위한 헤더 파일을 만드는 데 쓸 수 있다. 프로그램에 딸린 시험 루틴을 만드는 데도 쓸 수 있다. 프로그램과 그 헤더, 시험 루틴의 원본을 한데 두면 셋을 서로 어긋나지 않게 지키기가 쉬워진다. 이름 있는 절의 출력은 여러 출력 파일에 들어갈 수도 있다. @(bar1@>과 @(bar2@> 양쪽에서 @<foo@>를 부를 수 있기 때문이다.

@h $[Cc]$ CTANGLE이 모든 절의 중간 파트에 있는 #define 문장을 C 파일의 맨 앞이 아니라 지금 이 자리에 끼워 넣게 한다. 이를테면 매크로 정의가 include 파일 뒤에 오기를 바랄 때 쓸모 있다. (‘|...|’ 안에서는 CTANGLE이 무시한다.)

다음 몇 가지 제어 코드는 “제어 텍스트”를 여는데, 제어 텍스트는 다음 @>에서 끝난다. 닫는 @>는 제어 텍스트가 시작한 줄과 같은 줄에 있어야 한다. 이들 제어 코드에서 짝이 되는 @>까지의 문맥은 제한된 문맥이다.

@~ $[TMCc]$ * 뒤따르는 제어 텍스트가 다음 @>까지 C 프로그램의 식별자들과 함께 색인에 오른다. 이 텍스트는 로만체로 나온다. 이를테면 CWEB이 내놓는 색인에 “system dependencies”라는 어구를 넣으려면, 시스템 의존적이라고 색인에 올리고 싶은 절마다 @~system dependencies@>라고 치면 된다.

- Ⓞ. [TMCc] * 뒤따르는 제어 텍스트가 타자기 폴로 색인에 오른다.
- Ⓞ: [TMCc] * 뒤따르는 제어 텍스트가 T_EX 매크로 ‘\9’가 다스리는 모양으로 색인에 오른다. 이 매크로는 원하는 대로 정의하면 된다.
- Ⓞt [MCc] * 뒤따르는 제어 텍스트가 T_EX \hbox에 담겨 이웃한 C 프로그램과 함께 조판된다. 이 텍스트를 CTANGLE은 무시하지만 CWEAVE 안에서는 여러 가지로 쓸 수 있다. 이를테면 ‘|size < 20t\$^{15}\$0>|’라고 쳐서 ‘size < 2¹⁵’처럼 C와 고전적인 수학을 섞은 주석을 만들 수 있다.
- Ⓞ= [MCc] * 뒤따르는 제어 텍스트가 C 프로그램에 축자로 넘어간다.
- Ⓞq [LTMCC] * 뒤따르는 제어 텍스트는 깡그리 무시된다. CWEB 파일을 읽는 사람만을 위한 주석인 셈이다. Ⓞi로 limbo에 끼워 넣을 파일은 Ⓞq 주석으로 자기가 무엇인지 밝힐 수 있다. 또 다른 쓸모는 C 문자열 안에서 짝이 맞지 않는 괄호의 짝을 맞춰, 편집기의 괄호 맞추기가 갈피를 잃지 않게 하는 것이다.
- Ⓞ! [TMCc] * 색인에 오르는 식별자나 제어 텍스트 바로 앞에 ‘Ⓞ!’가 있으면 그 색인 항목의 절 번호에 밑줄이 그어진다. 이 관례는 식별자가 정의되거나 어떤 특별한 방식으로 설명되는 절을, 그저 쓰이는 절과 갈라 보이는 데 쓴다. 예약어나 한 글자 식별자는 밑줄 그어진 항목이 아니면 색인에 오르지 않는다. C 코드에서 식별자가 정의되거나 선언될 때는 CWEAVE가 ‘Ⓞ!’를 던지시 넣어 준다. 이를테면 정의

```
int array[max_dim], count = old_count;
```

는 *array*와 *count*라는 이름이 색인에서 밑줄 그어진 항목을 얻게 한다. 문장 레이블도, `main(int argc, char *argv[])` 같은 함수 정의도, `typedef` 정의도 밑줄을 뜻한다. (원형 없는) 옛 방식의 함수 정의는 그 인자를 정의하지 않는다. 다만 함수 몸통 앞에서 어느 방식대로 인자의 형이 선언되어 있으면 그 인자들도 정의된 것으로 여긴다 (즉 색인 항목에 밑줄이 그어진다). 이를테면 ‘`int argc; char *argv[]; { ... }`’가 그렇다. 그러므로 Ⓞ!가 필요한 일은 그리 잦지 않고, 유별난 구성이나 아래 같은 경우뿐이다.

```
enum boolean {Ⓞ!false, Ⓞ!true};
```

여기서는 Ⓞ!가 가장 좋은 결과를 준다. `enum`이 늘어놓는 낱말의 상수는 정의된 자리에서 색인에 저절로 밑줄이 그어지지 않기 때문이다.

이제 CTANGLE의 동작에만 영향을 주는 제어 코드로 넘어가자.

- Ⓞ' [MCc] 이 제어 코드는 CWEB과 본래의 WEB에서 뜻이 사뭇 달라 위험하다. CWEB에서는 길이가 1인 문자열의 ASCII 코드에 해당하는 십진 상수를 만든다 (이를테면 Ⓞ'a'는 97로, Ⓞ'\t'는 9로 CTANGLE된다). ASCII가 아닌 기계에서 ASCII로 작업해야 할 때 쓸 만하다. 하지만 대개는 문자 집합에 매이지 않는 프로그래밍에 <ctype.h>의 C 관례로 충분하다.
- Ⓞ& [MCc] Ⓞ& 연산은 C 출력에서 왼쪽에 있는 것과 오른쪽에 있는 것을 맞붙인다. 이 둘 사이에 빈칸이나 줄바꿈이 끼지 않는다.
- Ⓞl [L] CWEB 프로그래머는 T_EX 텍스트 안에서 흔히 금지되는 128-255 범위의 8비트 문자 코드를 마음대로 쓸 수 있다. 그런 문자는 문자열 안에서도, 심지어 C 프로그램의 식별자 안에서도 쓸 수 있다. 기본 ASCII 표준을 여러 모로 늘린 규격에서 위쪽 8비트 코드는 악센트 붙은 글자나 라틴이 아닌 문자 등에 해당한다. 그런 문자가 식별자에 나오면 CTANGLE은 올바른 C 코드를 만들어 내기 위해 그것을 표준 ASCII 영숫자나 _로 바꿔야 한다. 이 일은 음역 표로 하는데, 기본 설정에서는 ASCII 코드가 #ab인 문자에 Xab이라는 문자열을 찍지어 준다 (여기서 a와 b는 16진 숫자이고 a ≥ 8이다). limbo에 Ⓞl_{lab}newstring이라는 구성을 두면 CTANGLE에게 그 문자를 대신 newstring으로 바꾸라고 이르는 것이다. 이를테면 글자 ‘ü’의 ISO Latin-1 코드는 #FC (또는 ‘\374’)인데, 이것이 식별자에 나오면 CTANGLE은 보통 이 코드를 세 글자짜리 XFC로 바꾼다. Ⓞl fc ue라고 적어 두면 대신 ue로 음역된다.

CWEAVE는 8비트 문자를 음역하지 않고 T_EX에 그대로 넘긴다. 그러므로 T_EX 쪽이 그것을 받을 채비가 되어 있어야 한다. 표준이 아닌 식별자를 모두 “맞춤” 제어열로 조판하고 있다면, 그 문자들을 모두 T_EX이 글자로 여기게 해야 한다. 그렇지 않다면 8비트 코드를 T_EX에서 “활성”으로 만들거나, 필요한 특수 문자가 올바른 자리에 든 글꼴을 불러와야 한다. (식별자에는 T_EX 제어열 \it이 고르는 글꼴이 쓰인다.) 여러분의 언어를 위해 만들어진 CWEB 사용자용 특별 매크로 꾸러미를 찾아보라. 아니면, 용기가 있다면 손수 하나 지어 보라.

다음 여덟 가지 제어 코드('Ⓔ', 'Ⓔ/', 'Ⓔ!', 'Ⓔ#', 'Ⓔ+', 'Ⓔ;', 'Ⓔ[', 'Ⓔ')는 CTANGLE이 내놓는 C 프로그램에는 아무 영향도 주지 않는다. 이들은 유별난 상황에서 CWEAVE가 내놓는 T_EX 조판 C 코드를 읽기 좋게 다듬어 줄 뿐이다. CWEAVE에 불박인 조판 방식은 문법에 맞는 C 텍스트를 다룰 때는 꽤 훌륭하지만, 있을 수 있는 모든 경우를 감당하지는 못한다. 매크로와 절 이름이 섞인 텍스트 조각을 다뤄야 하는데 그런 조각이 반드시 C 문법을 따르지는 않기 때문이다. CWEB이 자동 조판을 눌러 이길 방법을 주기는 하지만, 가장 좋은 방법은 CWEAVE가 저절로 내놓은 결과를 보기 전에는 그런 것에 마음 쓰지 않는 것이다. 문서를 다듬을 즈음에는 아마 몇 군데만 손보면 될 테니까.

- Ⓔ, [MC] 이 제어 코드는 CWEAVE의 출력에 가는 빈칸을 넣는다. 매크로를 유별난 방식으로 쓸 때, 이를테면 식별자 둘이 맞붙어 있을 때 이 여분의 빈칸이 필요하기도 하다.
- Ⓔ/ [MC] 이 제어 코드는 CWEAVE가 조판한 C 프로그램 안에서 줄바꿈이 일어나게 한다. 줄바꿈은 99% 잘 들어맞는 방식에 따라 T_EX이 알아서 고르지만, 때로는 눈에 보이는 기준이 아니라 논리적인 기준으로 프로그램을 토막 내려고 줄바꿈을 억지로 넣고 싶을 것이다. 뒤에 주석이 오면 'Ⓔ/Ⓔ,'라고 적어 주석 앞에서 줄을 바꾸라.
- Ⓔ! [MC] 이 제어 코드는 식 한가운데에 줄을 바꿔도 좋은 자리를 정해 준다. 이를테면 대입문의 오른쪽에 긴 식이 있을 때, T_EX이 겉모습만 보고 고를 자리보다 더 논리적인 자리를 'Ⓔ!'로 일러 줄 수 있다.
- Ⓔ# [MC] 이 제어 코드는 Ⓔ/처럼 줄바꿈을 억지로 넣되, 그 자리의 줄 사이에 여분의 흰 공간도 조금 만든다. 이를테면 같은 절 안에 있지만 논리적으로는 갈라지는 매크로 정의 무리 사이에 쓸 만하다. CWEB은 함수와 함수 사이, 외부 선언과 함수 사이, 그리고 함수 안의 선언과 문장 사이에 이 여분의 공간을 저절로 넣는다.
- Ⓔ+ [MC] 이 제어 코드는 CWEAVE가 넣었을 줄바꿈을 물린다. 이를테면 짧은 if-else 구성을 한 줄에 두고 싶을 때 'else' 앞에서 그렇게 한다. 함수의 몸통인 복합문의 첫머리에 '{Ⓔ+'라고 적으면, 그 함수의 첫 선언이나 문장이 왼쪽 중괄호와 같은 줄에 나오고, 다음 줄의 둘째 선언이나 문장과 같은 만큼 들여 쓰인다.
- Ⓔ; [MC] 이 제어 코드는 조판을 위해서는 세미콜론처럼 다루어지되 보이지는 않는다. 이를테면 절이나 매크로가 대신하는 C 텍스트가 복합문이거나 세미콜론으로 끝날 때 그 절 이름이나 매크로 뒤에 쓸 수 있다. 다음 같은 구성을 생각해 보라.

```
if (condition) macro Ⓔ;
else break;
```

여기서 *macro*는 (중괄호로 감싼) 복합문으로 정의되어 있다. C 문법의 잘 알려진 알곳은 구석이다.

- Ⓔ[[MC] Ⓔ]를 보라.

- Ⓔ] [MC] CWEAVE가 식으로 조판했으면 하는 프로그램 본문을, 저절로 그렇게 되지 않는다면 Ⓔ[...Ⓔ] 괄호로 감싸라. (유별난 매크로 인자에 이따금 해당한다.) 또한 함수를 가리키는 포인터를 선언할 때는 단순한 형 이름과 왼쪽 괄호 사이에 'Ⓔ[Ⓔ]'를 넣으라. 이를테면

```
int Ⓔ[Ⓔ] (*f)();
```

처럼. 그러지 않으면 CWEAVE는 그 선언의 앞부분을 C++ 식 'int(*f)'와 헷갈린다. 또 다른 보기로, C 코드 한가운데에 낮은 수준의 #define 명령을 쓰고 싶는데 그 정의가 캐스트로 시작하는 경우가 있다.

```
#define foo Ⓔ[(int)(bar)Ⓔ]
```

남은 제어 코드는 CWEB이 보는 입력을 다스린다.

- Ⓔx Ⓔy Ⓔz [*change_file*] CWEAVE와 CTANGLE은 *web_file*과 *change_file*이라는 두 입력 파일로 일하도록 만들어졌다. *change_file*에는 *web_file*의 고른 부분을 눌러 이길 자료가 들어 있다. 그렇게 합쳐진 텍스트가 사실 이 글의 다른 곳에서 CWEB 파일이라고 부른 것이다.

그 열개는 이렇다. 변경 파일은 “변경”을 0개 이상 담는데, 변경 하나는 ‘Ⓔx<옛 줄들>Ⓔy<새 줄들>Ⓔz’의 꼴이다. 변경 파일에서만 쓸 수 있는 특별한 제어 코드 Ⓔx, Ⓔy, Ⓔz는 줄의 첫머리에 와야 하고, 그 줄의 나머지는 무시된다. <옛 줄들>은 *web_file*의 이어지는 줄들과 정확히 맞는 재료이고, <새 줄들>은 옛것을 대신할 0개 이상의 줄이다. 어떤 변경의 첫 “옛 줄”이 *web_file*의 어느 줄과 맞으면, 그 변경의 나머지 줄도 모두 맞아야 한다.

변경과 변경 사이, 첫 변경 앞, 마지막 변경 뒤에는 ‘Ⓔx’나 ‘Ⓔy’, ‘Ⓔz’로 시작하지 않는 줄이 몇 개든 있어도 된다. 그런 줄은 그냥 지나쳐지고 맞춰 보는 데 쓰이지 않는다.

이 두 입력 기능은 다른 데서 받은 으뜸 CWEB 파일(이를테면 `tangle.w`나 `weave.w`, `tex.web`)을 다룰 때, 그 프로그램을 자기 컴퓨터 체계에 맞게 고치고 싶을 때 쓸모 있다. 으뜸 web 파일을 망가뜨리지 않고 체계에 딸린 변경을 디버깅할 수 있고, 변경이 제대로 돌아가게 되면 이따금 받게 될 으뜸 web 파일의 새 판에도 손쉽게 엮을 수 있다.

@i [*web_file*] 더 나아가 *web_file* 자체를 여러 파일을 합친 것으로 할 수도 있다. CWEAVE든 CTANGLE이든 파일을 읽다가 줄의 첫머리에서 제어 코드 @i를 만나면, 읽던 것을 멈추고 @i 뒤에 적힌 이름의 파일을 보기 시작한다. C 전처리가 #include 줄을 만났을 때와 사뭇 비슷하다. 끼워 넣은 파일을 끝까지 읽고 나면 원래 파일의 다음 줄로 돌아간다. @i 뒤의 파일 이름은 " 문자로 감싸도 되지만 그 울타리는 있어도 그만 없어도 그만이다. 끼워 넣기는 겹쳐도 된다.

변경 파일에도 @i로 시작하는 줄을 둘 수 있다. 이렇게 하면 끼워 넣은 파일 하나를 다른 것으로 갈아치울 수 있다. 개념상으로는 위에서 말한 대체 장치가 먼저 일하고, 그 결과에서 @i 줄을 찾는다. 변경 파일의 @y와 @z 사이에 @i foo가 있으면 foo 파일과 그것이 끼워 넣는 파일들의 낱말의 줄은 바꿀 수 없다. 다만 바뀌지 않은 입력이 끼워 넣은 파일의 줄에는 변경을 가할 수 있다.

UNIX 체계(그리고 환경 변수를 지원하는 다른 체계)에서는, 환경 변수 CWEBINPUTS가 설정되어 있거나 같은 이름의 컴파일러 플래그가 컴파일 때 정의되어 있으면, CWEB은 끼워 넣을 파일을 현재 디렉터리에서 찾지 못했을 때 그 이름의 디렉터리에서 찾는다.

그 밖의 기능과 유의할 점

1. 확장 문자 집합을 갖춘 어떤 CWEB 설치에서는 문자 ‘↑’, ‘↓’, ‘←’, ‘→’, ‘≠’, ‘≤’, ‘≥’, ‘≡’, ‘√’, ‘∧’, ‘C’, ‘∩’ 을 각각 ‘++’, ‘--’, ‘->’, ‘!=’, ‘<=’, ‘>=’, ‘==’, ‘||’, ‘&&’, ‘<<’, ‘>>’의 줄임으로 칠 수 있다.

2. 확장 문자 집합이 있다면 위 @1 규칙에서 말한 대로 아주 적은 제약만 지키고 쓸 수 있다. 하지만 확장 문자 집합이 없는 세상의 딱한 이들 모두에게 쓸모 있는 프로그램을 쓰고 싶다면 표준 ASCII 문자만 고집하는 게 좋다.

3. CWEAVE가 내놓는 TeX 파일은 한 줄이 최대 80글자가 되도록 잘린다. TeX 텍스트를 베낄 때는 원래 있던 줄바꿈도 함께 베껴진다. 너무 알맞은 것만 하지 않는다면, CWEAVE는 TeX 주석이 두 줄 이상으로 갈리는 때를 알아차리고 그렇게 이어지는 주석의 첫머리에 ‘%’를 붙여 준다.

4. C 텍스트는 “상향식” 절차로 옮겨진다. 각 토큰을 “품사”로 알아본 다음, CWEAVE의 문서에 설명된 특별한 문법에 따라 품사들을 될 수 있는 한 점점 더 큰 어구로 묶어 나가는 것이다. 낱말의 식별자나 짧은 식 같은 단순한 구성이 어떻게 옮겨지는지는 CWEAVE가 하는 일의 보기 몇 개만 봐도 쉽게 익힐 수 있다. 그러나 전체 열개는 C 자체보다 훨씬 많은 것을 감당해야 하므로 다소 복잡하다. 게다가 출력에는 쓰인 글꼴과 바라는 쪽 너비에 따라 TeX이 필요한 만큼 들여 쓰고 줄을 바꾸게 하는 코드가 박혀 있다. 그러니 긴 C 텍스트를 |...|로 감싸는 것은 피하는 게 좋다. |...|안의 텍스트를 C에서 TeX으로 옮길 때는 들여쓰기와 줄바꿈 코드가 빠지기 때문이다. 단순한 식이나 문장만 담으라. C 전처리기 명령을 |...|로 감싸려면 그것을 여는 #이 줄의 첫머리에 있어야 한다. 그러지 않으면 CWEAVE가 제대로 찍지 못한다.

5. |...|텍스트 안에는 주석을 둘 수 없다. ‘|’가 TeX 텍스트에서 C 텍스트로 바뀔을 알리고 나면, 문자열이나 제어 텍스트, 절 이름의 일부가 아닌 다음 ‘|’가 그 C 텍스트를 끝낸다.

6. 주석 안에서는 왼쪽과 오른쪽 중괄호가 제대로 짝지어 겹쳐야 한다. 그러지 않으면 CWEAVE가 불평한다. 그래도 TeX이 너무 해매지 않도록 중괄호의 짝을 맞춰 보려고 애쓰기는 한다.

7. 프로그램을 디버깅하다가 C 코드의 일부를 빼기로 했다면, 그것을 그냥 “주석 처리”하지 말라. 그런 주석은 CWEB 문서의 정신에 맞지 않는다. 읽는 이에게는 주석 처리되지 않은 명령을 설명하는 글처럼 보일 테니 말이다. 게다가 프로그램의 주석은 올바른 TeX 텍스트여야 한다. 그러니 C 문장을 /*|...|*/가 아니라 /*...*/로 감싸면 CWEAVE가 갈피를 잃는다. 꼭 C 코드를 주석 처리해야 한다면 #if 0==1과 #endif 같은 전처리기 명령으로 감싸면 된다.

8. @f 기능은 한 식별자가 다른 식별자처럼 행동하도록 정의하게 해 주는데, 이 형식 정의는 차례대로 이루어진다. 대체로 한 식별자는 문서 전체에서 찍히는 모양이 하나뿐이고, 그 모양은 그것을 정의하는 @f보다 앞에서도 쓰인다. 까닭은 CWEAVE가 두 번에 걸쳐 훑기 때문이다. 첫 훑기에서 @f와 상호 참조를 처리하고 둘째 훑기에서 출력을 낸다. (다만 typedef 선언 덕분에 넘지시 볼드 모양을 얻는 식별자는 이 규칙을 따르지 않는다. 그런 식별자는 해당

`typedef`의 앞뒤에서 다르게 적힌다. 아쉽지만 고치기 어렵다. `typedef`의 앞이나 뒤에 이를테면 ‘@s foo int’ 라고 적으면 이 제약을 비켜 갈 수 있다.)

9. 조판된 C 코드에 ‘@,’가 주는 가는 빈칸보다 더 자유로운 간격을 넣고 싶을 때가 있다. @t 기능을 그 일에 쓸 수 있다. 이를테면 ‘@t\hskip 1in@>’은 1인치의 빈 곳을 남긴다. 또한 ‘@t\4@>’로 들여쓰기 한 단위만큼 물러설 수 있다. 제어열 \4가 `cwebmac`에 그런 물러섬으로 정의되어 있기 때문이다. (이 제어열은 이를테면 레이블이 붙은 문장이 있는 줄의 첫머리에 쓰여, 레이블이 왼쪽으로 조금 비어져 나오게 한다.) 식 한가운데에서 줄을 바꾸도록 ‘@t}\3{-5@>’를 쓸 수도 있다.

10. CWEB에서 각 식별자는 조판 관례를 하나만 갖는다. 그러므로 이를테면 형 이름과 `struct`의 멤버 이름으로 같은 식별자를 함께 쓰지는 말아야 한다. C이 그것을 허락하더라도 그렇다.

프로그램 돌리기

CTANGLE의 UNIX 명령줄은

```
ctangle [options] web_file[.w] [{change_file[.ch]}|-] [out_file]
```

이고 CWEB에도 같은 관례가 적용된다. ‘-’를 적거나 변경 파일을 적지 않으면 변경 파일은 비어 있는 것으로 친다. 확장자 `.w`와 `.ch`는 주어진 파일 이름에 점이 없을 때만 붙는다. 그렇게 정해진 web 파일을 찾지 못하면 확장자 `.web`으로도 찾아본다. 이를테면 ‘cweave cob’은 `cob.w`를 읽으려 하고, 그것이 없으면 포기하기 전에 `cob.web`을 찾아본다. 출력 파일 이름을 적지 않으면 CTANGLE이 내놓는 C 파일의 이름은 확장자 `.c`를 붙여 얻고, CWEB이 내놓는 `TeX` 파일의 이름에는 확장자 `.tex`이 붙는다. CWEB이 내놓는 색인 파일은 `.tex` 자리에 `.idx`와 `.scn`이 온다.

짧게 치기를 좋아하는 프로그래머라면 셸에서 ‘wv’가 ‘cweave -bhp’로 펼쳐지도록 해 두어도 좋다. 그러면 오류 메시지를 뺀 CWEB의 단말 출력이 거의 사라진다.

옵션은 끄려면 - 기호로, 켜려면 + 기호로 연다. 이를테면 ‘-fb’는 옵션 `f`와 `b`를 끄고, ‘+s’는 옵션 `s`를 켜다. 옵션은 파일 이름 앞에 두어도 되고 뒤에 두어도 되며 양쪽에 두어도 된다. 지금 구현된 옵션은 다음과 같다.

- b 실행을 시작할 때 배너 줄을 찍는다. (기본으로 켜져 있다.)
- e CWEB이 조판한 C 재료를 `\PB{...}` 괄호로 감싸, 특별한 고리를 걸 수 있게 한다. (기본으로 꺼져 있다. CTANGLE에는 영향이 없다.)
- f CWEB이 조판하는 C 문장마다 뒤에 줄바꿈을 억지로 넣는다. (기본으로 켜져 있다. -f는 종이를 아끼지만 어떤 이들에게는 덜 C답게 보인다.) (CTANGLE에는 영향이 없다.)
- h 무사히 끝났을 때 기쁜 소식을 찍는다. (기본으로 켜져 있다.)
- p 프로그램이 도는 동안 진행 상황을 알린다. (기본으로 켜져 있다.)
- s 프로그램이 끝까지 돈 뒤 메모리 사용에 대한 통계를 보여 준다. (기본으로 꺼져 있다.) CWEB 파일이나 절이 크다면 CTANGLE이나 CWEB의 용량에 얼마나 가까이 다가섰는지 볼 필요가 있을 것이다.
- x CWEB이 내놓는 `TeX` 파일에 색인과 목차를 넣는다. (기본으로 켜져 있다.) (CTANGLE에는 영향이 없다.)

조판에 대한 좀 더 자세한 이야기

CWEB이 어떤 상황을 다루는 방식이 마음에 들지 않을 수도 있다. 정 아쉽다면 문법을 바꿔 CWEB을 손볼 수 있다. 그것은 소스 코드를 고친다는 뜻인데, 아마 재미있는 일이 될 것이다. 문법 규칙 표는 CWEB 소스 목록에 실려 있고, CWEB 소스에 있는 `prod.w` 파일을 베끼고 다음 ‘cweave -x prod’와 ‘tex prod’를 차례로 돌리면 그 표만 따로 뽑아 볼 수 있다.

여러분의 C 코드를 CWEB이 어떻게 파싱하는지는 그 앞에 ‘@ @c @2’ 줄을 두면 낱낱이 볼 수 있다. (제어 코드 ‘@2’는 “옛보기” 모드를 켜고 ‘@0’은 끈다.) 이를테면 다음 파일에 CWEB을 돌리면

```
@ @c @2
main (argc,argv)
char **argv;
{ for (;argc>0;argc--) printf("%s\n",argv[argc-1]); }
```

화면에 다음 같은 알쏭달쏭한 것이 나온다.

```
[...]
14:*exp +( + exp...
11:*exp +exp+ raw...
10:*+exp+ raw +ubinop?+...
[...]
60: +fn_decl+*+{+ -stmt- +}-
55:*+fn_decl+ -stmt-
52:*+function-
[...]
```

첫 줄은 문법 규칙 14가 방금 적용되었고, CWEB의 기억 속에는 지금 *exp*(식), 여는 괄호, 다시 *exp*, 닫는 괄호에 해당하는 TeX 코드 덩어리 (“scrap”이라고 부른다)들이 차례로 놓여 있으며, 그 뒤로 파서가 아직 보지 않은 scrap이 더 있다는 말이다. (+와 - 기호는 scrap의 경계에서 TeX이 수학 모드 안에 있어야 하는지 밖에 있어야 하는지를 정한다. *는 파서가 지금 있는 자리를 보여 준다.) 이어서 규칙 11이 적용되어 (*exp*)가 하나의 *exp*가 되고, 이런 식으로 나아간다. 끝에 가서는 C 텍스트 전체가 *function* 형의 커다란 scrap 하나가 된다.

일이 늘 그렇게 매끄럽지는 않아서 여러 줄이 한 덩어리로 뭉쳐 나오기도 한다. 이는 CWEB이 여러분의 C 코드에서 무언가를 식이지 못했다는 뜻이다. 이를테면 앞의 프로그램에서 ‘char **argv;’가 있던 자리에 대신 ‘@<Argument definitions>’가 있었다고 하자. 그러면 CWEB은 절 이름을 그저 *exp*로 여기므로 조금 어리둥절했을 것이다. 그래서 ‘<Argument declarations 2>’를 ‘main(argc, argv)’와 같은 줄에 조판하라고 TeX에게 일렀을 것이다. 그럴 때는 ‘main(argc, argv)’ 뒤에 ‘@/’를 두어 CWEB을 도와야 한다.

CWEB은 선언과 블록의 첫 문장으로 보이는 것 사이에 여분의 공간을 조금 저절로 넣는다. 그 간격을 그 자리에서만 물리치는 한 가지 방법은

```
int x;@+@t}\6{@>
@<Other locals>@;@#
```

이다. 여기서 ‘@#’이 ‘<Other locals>’ 뒤에 여분의 공간을 넣어 준다.

하이퍼텍스트와 하이퍼문서

CWEB과 월드 와이드 웹 사이의 닮은 점을 알아챈 사람이 물론 많다. 실제로 CWEB 매크로는 CWEB의 출력을 손쉽게 Portable Document Format으로 바꿀 수 있게 짜여 있다. Mark A. Wicks가 만든, 널리 쓰이는 공개 소스 프로그램 *dvipdfm*을 쓰면 Adobe Acrobat Reader로 읽을 수 있고 눌러서 따라갈 수 있는 하이퍼링크가 붙은 문서가 나온다. CWEB로 *cob.w*를 *cob.tex*으로 바꾼 다음, TeX을 어느 방식으로 부르는 대신 다음 명령을 주면 프로그램의 하이퍼텍스트 판을 만들어 볼 수 있다.

```
tex "\let\pdf+ \input cob"
dvipdfm cob
acroread cob.pdf
```

(이것이 되게 해 주는 매크로를 만들어 준 Hans Hagen과 César Augusto Rorato Crusius, Julian Gilbey에게 고마움을 전한다.) 아니면 Hàn Thé Thành과 Andreas Scherer 덕분에 ‘*pdftex cob*’이라고만 해도 한 걸음에 *cob.pdf*를 만들 수 있다.

식별자가 쓰인 자리에서 정의된 자리로 가는 링크를 만들어 CWEB의 어느 상호 참조를 넘긴 CTWILL이라는 좀 더 공들인 체계도 있다. 다만 식별자가 여러 번 정의된 경우에는 손이 조금 더 가는 것을 받아들여야 한다. CTWILL은 본디 종이 출력을 겨냥한 것이지만 그 원리는 하이퍼텍스트에도 쓸 수 있다. D. E. Knuth의 *Digital Typography* (1999) 11장과 <ftp://ftp.cs.stanford.edu/pub/ctwill>의 프로그램 소스를 보라.

부록

CWEB으로 쓴 실제 프로그램의 보기로, 부록 A에는 CWEB 프로그램 자체에서 뽑은 대목이 실려 있다. 이 부록의 목록을 찬찬히 살펴보는 독자는 CWEB의 기본 생각을 잘 느끼게 될 것이다.

부록 B는 **CWEAVE**의 출력을 받아들이도록 **T_EX**을 채비하는 파일이고, 부록 C는 그 매크로 가운데 몇을 써서 출력 모양을 바꾸는 방법을 이야기한다.

CWEB 소스에서 UNIX 명령 `make fullmanual`로 만들 수 있는 이 설명서의 “긴” 판에는 **CTANGLE**과 **CWEAVE**의 온전한 소스 코드를 보여 주는 부록 D, E, F도 실려 있다.

부록 A: CWEB 프로그램에서 뽑은 대목

이 부록은 네 개의 목록으로 이루어진다. 첫째 것은 CWEAVE와 CTANGLE이 함께 쓰는 루틴이 담긴 `common.w` 파일의 12-15 절을 만들어 낸 CWEB 입력을 보여 준다. 프로그램의 구조를 보이려고 몇몇 줄을 들여 썼음에 주목하라. 들여쓰기는 CWEAVE와 CTANGLE이 무시하지만, 그렇게 들여 쓰면 CWEB 파일이 사뭇 읽기 좋다는 것을 사용자들은 알고 있다.

둘째와 셋째 목록은 `common.w`에 프로그램을 돌렸을 때 CTANGLE이 내놓는 C 코드와 CWEAVE가 내놓는 $\text{\TeX}$ 코드의 해당 부분을 보여 준다. 넷째 목록은 그 출력을 찍었을 때 어떻게 보이는지를 보여 준다.

```
@ Procedure |prime_the_change_buffer|
sets |change_buffer| in preparation for the next matching operation.
Since blank lines in the change file are not used for matching, we have
|(change_limit==change_buffer && !changing)| if and only if
the change file is exhausted. This procedure is called only when
|changing| is 1; hence error messages will be reported correctly.

@c
void
prime_the_change_buffer()
{
    change_limit=change_buffer; /* this value is used if the change file ends */
    @<Skip over comment lines in the change file; |return| if end of file@>;
    @<Skip to the next nonblank line; |return| if end of file@>;
    @<Move |buffer| and |limit| to |change_buffer| and |change_limit|@>;
}

@ While looking for a line that begins with \.{@x} in the change file, we
allow lines that begin with \.{@y}, as long as they don't begin with \.{@y},
\.{@z}, or \.{@i} (which would probably mean that the change file is fouled up).

@<Skip over comment lines in the change file...@>=
while(1) {
    change_line++;
    if (!input_ln(change_file)) return;
    if (limit<buffer+2) continue;
    if (buffer[0]!='@') continue;
    if (xisupper(buffer[1])) buffer[1]=tolower(buffer[1]);
    if (buffer[1]=='x') break;
    if (buffer[1]=='y' || buffer[1]=='z' || buffer[1]=='i') {
        loc=buffer+2;
        err_print("!! Missing @x in change file");
    }
    @.Missing @x...@>
}

@ Here we are looking at lines following the \.{@x}.

@<Skip to the next nonblank line...@>=
do {
    change_line++;
    if (!input_ln(change_file)) {
        err_print("!! Change file ended after @x");
    }
    @.Change file ended...@>
    return;
} while (limit==buffer);

@ @<Move |buffer| and |limit| to |change_buffer| and |change_limit|@>=
{
    change_limit=change_buffer-buffer+limit;
    strncpy(change_buffer,buffer,limit-buffer+1);
}
```

앞쪽의 소스에 해당하는, CTANGLE이 만들어 낸 C 코드의 일부다. 13, 14, 15 절이 12절 안으로 엉켜 들어갔음 (tangle)에 주목하라.

```

/*:9*//*12:*/
#line 247 "common.w"

void
prime_the_change_buffer()
{
  change_limit= change_buffer;
/*13:*/
#line 261 "common.w"

  while(1){
    change_line++;
    if(!input_ln(change_file))return;
    if(limit<buffer+2)continue;
    if(buffer[0]!='@')continue;
    if(xisupper(buffer[1]))buffer[1]= tolower(buffer[1]);
    if(buffer[1]=='x')break;
    if(buffer[1]=='y' || buffer[1]=='z' || buffer[1]=='i'){
      loc= buffer+2;
      err_print("! Missing @x in change file");
    }
  }

/*:13*/
#line 252 "common.w"
;
/*14:*/
#line 278 "common.w"

do{
  change_line++;
  if(!input_ln(change_file)){
    err_print("! Change file ended after @x");

    return;
  }
}while(limit==buffer);

/*:14*/
#line 253 "common.w"
;
/*15:*/
#line 288 "common.w"

{
  change_limit= change_buffer-buffer+limit;
  strncpy(change_buffer,buffer,limit-buffer+1);
}

/*:15*/
#line 254 "common.w"
;
}

/*:12*//*16:*/

```

common.tex에서 그에 해당하는 대목이다.

```

\M{12}Procedure \PB{\{\prime\_the\_change\_buffer\}}
sets \PB{\{\change\_buffer\}} in preparation for the next matching operation.
Since blank lines in the change file are not used for matching, we have
\PB{\{\change\_limit\}E\{\change\_buffer\}W\R\{\changing\}}$} if and only if
the change file is exhausted. This procedure is called only when
\PB{\{\changing\}} is 1; hence error messages will be reported correctly.

\Y\B&\{void\} \{\prime\_the\_change\_buffer\}(\,)\1\1\2\2\6
$\}\{\}\$1\6
$\}\{\change\_limit\}K\{\change\_buffer\}\}$;C{ this value is used if the
change file ends }\6
\X13:Skip over comment lines in the change file; \PB{\&\{return\}} if end of file%
\X;\6
\X14:Skip to the next nonblank line; \PB{\&\{return\}} if end of file\X;\6
\X15:Move \PB{\{\buffer\}} and \PB{\{\limit\}} to \PB{\{\change\_buffer\}} and %
\PB{\{\change\_limit\}}\X;\6
\4$\}\{\}\$2\par
\fi

\M{13}While looking for a line that begins with \.{@x} in the change file, we
allow lines that begin with \.{@}, as long as they don't begin with \.{@y},
\.{@z}, or \.{@i} (which would probably mean that the change file is fouled
up).

\Y\B\4\X13:Skip over comment lines in the change file; \PB{\&\{return\}} if end
of file\X$\}\E{\}$\6
&\{while\} (\T{1})\5
$\}\{\}\$1\6
$\}\{\change\_line\}PP;\}$\6
&\{if\} $\}(\R\{\input\_ln\}(\{\change\_file\}))\}$\1\5
&\{return\};\2\6
&\{if\} $\}(\{\limit\}<\{\buffer\}+\T{2})\}\}$\1\5
&\{continue\};\2\6
&\{if\} $\}(\{\buffer\}[\T{0}]\I\.{'}\})\}$\1\5
&\{continue\};\2\6
&\{if\} (\{\xisupper\}(\{\buffer\}[\T{1}]))\1\5
$\}\{\buffer\}[\T{1}]K\{\tolower\}(\{\buffer\}[\T{1}]);\}$\2\6
&\{if\} $\}(\{\buffer\}[\T{1}]\E\.{'}x')\}\}$\1\5
&\{break\};\2\6
&\{if\} $\}(\{\buffer\}[\T{1}]\E\.{'}y'\}V\{\buffer\}[\T{1}]\E\.{'}z'\}V\{\buffer\}[%
\T{1}]\E\.{'}i')\}\}$\5
$\}\{\}\$1\6
$\}\{\loc\}K\{\buffer\}+\T{2};\}$\6
\{\err\_print\}(\.{'}!\ Missing\ @x\ in\ cha\)\.\nge\ file");\6
\4$\}\{\}\$2\6
\4$\}\{\}\$2\par
\U12.\fi

\M{14}Here we are looking at lines following the \.{@x}.

\Y\B\4\X14:Skip to the next nonblank line; \PB{\&\{return\}} if end of file\X$\}%
\E{\}$\6
&\{do\}\5
$\}\{\}\$1\6
$\}\{\change\_line\}PP;\}$\6
&\{if\} $\}(\R\{\input\_ln\}(\{\change\_file\}))\}\}$\5
$\}\{\}\$1\6
\{\err\_print\}(\.{'}!\ Change\ file\ ended\)\.\{ after\ @x");\6
&\{return\};\6
\4$\}\{\}\$2\6
\4$\}\{\}\$2\5
&\{while\} $\}(\{\limit\}E\{\buffer\})\}$;\par
\U12.\fi

\M{15}\B\X15:Move \PB{\{\buffer\}} and \PB{\{\limit\}} to \PB{\{\change\_buffer\}}
and \PB{\{\change\_limit\}}\X$\}\E{\}$\6
$\}\{\}\$1\6
$\}\{\change\_limit\}K\{\change\_buffer\}-\{\buffer\}+\{\limit\};\}$\6
$\}\{\strncpy\}(\{\change\_buffer\},\39\{\buffer\},\39\{\limit\}-\{\buffer\}+%
\T{1});\}$\6
\4$\}\{\}\$2\par
\Us12\ET16.\fi

```


그리고 같은 대목을 조판하면 이렇게 보인다. And here's what the same excerpt looks like when typeset.

12. Procedure *prime_the_change_buffer* sets *change_buffer* in preparation for the next matching operation. Since blank lines in the change file are not used for matching, we have ($change_limit \equiv change_buffer \wedge \neg changing$) if and only if the change file is exhausted. This procedure is called only when *changing* is 1; hence error messages will be reported correctly.

```
void prime_the_change_buffer()
{
    change_limit = change_buffer;    /* this value is used if the change file ends */
    < Skip over comment lines in the change file; return if end of file 13 >;
    < Skip to the next nonblank line; return if end of file 14 >;
    < Move buffer and limit to change_buffer and change_limit 15 >;
}
```

13. While looking for a line that begins with *@x* in the change file, we allow lines that begin with *@*, as long as they don't begin with *@y*, *@z*, or *@i* (which would probably mean that the change file is fouled up).

< Skip over comment lines in the change file; **return** if end of file 13 > $\equiv$

```
while (1) {
    change_line++;
    if ( $\neg input\_ln(change\_file)$ ) return;
    if ( $limit < buffer + 2$ ) continue;
    if ( $buffer[0] \neq '@'$ ) continue;
    if ( $xisupper(buffer[1])$ )  $buffer[1] = tolower(buffer[1])$ ;
    if ( $buffer[1] \equiv 'x'$ ) break;
    if ( $buffer[1] \equiv 'y' \vee buffer[1] \equiv 'z' \vee buffer[1] \equiv 'i'$ ) {
        loc = buffer + 2;
        err_print("! Missing @x in change file");
    }
}
```

이 코드는 12 절에서 쓰인다.

14. Here we are looking at lines following the *@x*.

< Skip to the next nonblank line; **return** if end of file 14 > $\equiv$

```
do {
    change_line++;
    if ( $\neg input\_ln(change\_file)$ ) {
        err_print("! Change file ended after @x");
        return;
    }
} while ( $limit \equiv buffer$ );
```

이 코드는 12 절에서 쓰인다.

15. < Move *buffer* and *limit* to *change_buffer* and *change_limit* 15 > $\equiv$

```
{
    change_limit = change_buffer - buffer + limit;
    strncpy(change_buffer, buffer, limit - buffer + 1);
}
```

이 코드는 12, 16 절에서 쓰인다.

부록 B: cwebmac.tex 파일

CWEAVE의 출력에 필요한 기능을 뒷받침하려고 “plain TeX” 형식을 넓히는 파일이다.

```
% standard macros for CWEB listings (in addition to plain.tex)
% Version 3.70 --- July 2017
\ifx\renewenvironment\undefined\else\endinput\fi % LaTeX will use other macros
\edef\fmtversion{\fmtversion+CWEB3.70}
\chardef\cwebversion=3 \chardef\cwebrevision=70
\newif\ifpdf
\ifx\pdf+\pdftrue\fi
% Uncomment the following line if you want PDF goodies to be the default
%\ifx\pdf-\else\pdftrue\fi
\def\pdflinkcolor{0 0 1} % the RGB values for hyperlink color
\newif\ifpdfTeX
\ifx\pdfoutput\undefined \pdfTeXfalse \else\ifnum\pdfoutput=0 \pdfTeXfalse
%\else \pdfTeXtrue \pdfoutput=1 \input pdfcolor \let\setcolor\pdfsetcolor \fi\fi
\else \pdfTeXtrue \pdfoutput=1 % changed in 3.69
\def\Black{\pdfliteral{0 g 0 G}} % use rgb colors for direct PDF output too
\def\Blue{\pdfliteral{0 0 1 rg 0 0 1 RG}}
\fi\fi
\newif\ifacro \ifpdf\acrotrue\fi \ifpdfTeX\acrotrue\fi

\let\:=\ . % preserve a way to get the dot accent
% (all other accents will still work as usual)

\parskip 0pt % no stretch between paragraphs
\parindent 1em % for paragraphs and for the first line of C text

\font\ninerm=cmr9
\let\mc=\ninerm % medium caps
\def\CEE/{\{\mc C\spacefactor1000}}
\def\UNIX/{\{\mc U\kern-.05emNIX\spacefactor1000}}
\def\TEX/{\TeX}
\def\CPLUSPLUS/{\{\mc C\PP\spacefactor1000}}
\def\Cee{\CEE/} % for backward compatibility
\def\9#1{
% with this definition of \9 you can say @:sort key}{TeX code@>
% to alphabetize an index entry by the sort key but format with the TeX code
\font\eightrm=cmr8
\let\sc=\eightrm % for smallish caps (NOT a caps-and-small-caps font)
\let\mainfont=\tenrm
\let\cmntfont\tenrm
%\font\tenss=cmss10 \let\cmntfont\tenss % alternative comment font
\font\titelfont=cmr7 scaled\magstep4 % title on the contents page
\font\titelfont=cmntt10 scaled\magstep2 % typewriter type in title
\font\tentex=cmtex10 % TeX extended character set (used in strings)
\fontdimen7\tentex=0pt % no double space after sentences

\def\#1{\leavevmode\hbox{\it#1\kern.05em}} % italic type for identifiers
\def\|1{\leavevmode\hbox{\$#1\$}} % one-letter identifiers look better this way
\def\&#1{\leavevmode\hbox{\bf
\def\_ {\kern.04em\vbox{\hrule width.3em height .6pt}\kern.08em}%
#1\kern.05em}} % boldface type for reserved words
\def\.#1{\leavevmode\hbox{\tentex % typewriter type for strings
\let\=\BS % backslash in a string
\let\{=\LB % left brace in a string
\let\}=\RB % right brace in a string
\let\~=\TL % tilde in a string
\let\ =\SP % space in a string
\let\_=\UL % underline in a string
\let\&=\AM % ampersand in a string
\let\^=\CF % circumflex in a string
#1\kern.05em}}
\def\){\tentex\kern-.05em}\discretionary{\hbox{\tentex\BS}}{\}{}}
\def\AT{0} % at sign for control text (not needed in versions >= 2.9)
\def\ATL{\par\noindent\bgroup\catcode'\_ =12 \postATL} % print @l in limbo
\def\postATL#1 #2 {\bf letter \{\uppercase{\char"#1}}
\tangles as \tentex "#2"\egroup\par}
\def\noATL#1 #2 {}
\def\noat1{\let\ATL=\noATL} % suppress output from @l
```

```

\def\ATH{\acrofalse\X\kern-.5em:Preprocessor definitions\X}
\let\PB=\relax % hook for program brackets [...] in TeX part or section name

\chardef\AM='& % ampersand character in a string
\chardef\BS='\ % backslash in a string
\chardef\LB='{ % left brace in a string
\chardef\RB='} % right brace in a string
\def\SP{\ttchar\ } % (visible) space in a string
\chardef\TL='~ % tilde in a string
\chardef\UL='_ % underline character in a string
\chardef\CF='^ % circumflex character in a string

\newbox\PPbox % symbol for ++
\setbox\PPbox=\hbox{\kern.5pt\raise1pt\hbox{\sevenrm+\kern-1pt+}\kern.5pt}
\def\PP{\copy\PPbox}
\newbox\MMbox \setbox\MMbox=\hbox{\kern.5pt\raise1pt\hbox{\sevensy\char0
\kern-1pt\char0}\kern.5pt}
\def\MM{\copy\MMbox}
\newbox\MGbox % symbol for ->
\setbox\MGbox=\hbox{\kern-2pt\lower3pt\hbox{\teni\char'176}\kern1pt}
\def\MG{\copy\MGbox}
\def\MRL#1{\mathrel{\let\K==#1}}
%\def\MRL#1{\KK#1}\def\KK#1#2{\buildrel\;#1\over{#2}}
\let\GG=\gg
\let\LL=\ll
\let\NULL=\Lambda
\mathchardef\AND="2026 % bitwise and; also \& (unary operator)
\let\OR=\mid % bitwise or
\let\XOR=\oplus % bitwise exclusive or
\def\CM{\sim} % bitwise complement
\newbox\MODbox \setbox\MODbox=\hbox{\eightrm\%}
\def\MOD{\mathbin{\copy\MODbox}}
\def\DC{\kern.1em::\kern.1em} % symbol for ::
\def\PA{\mathbin{.*}} % symbol for .*
\def\MGA{\mathbin{\MG*}} % symbol for ->*
\def\this{\&\this}

\newbox\bak \setbox\bak=\hbox to -1em{} % backspace one em
\newbox\bakk \setbox\bakk=\hbox to -2em{} % backspace two ems

\newcount\ind % current indentation in ems
\def\1{\global\advance\ind by1\hangindent\ind em} % indent one more notch
\def\2{\global\advance\ind by-1} % indent one less notch
\def\3#1{\hfil\penalty#10\hfilneg} % optional break within a statement
\def\4{\copy\bak} % backspace one notch
\def\5{\hfil\penalty-1\hfilneg\kern2.5em\copy\bakk\ignorespaces}% optional break
\def\6{\ifmmode\else\par % forced break
\hangindent\ind em\noindent\kern\ind em\copy\bakk\ignorespaces\fi}
\def\7{\Y\6} % forced break and a little extra space
\def\8{\hskip-\ind em\hskip 2em} % no indentation

\newcount\gdepth % depth of current major group, plus one
\newcount\secpagedepth
\secpagedepth=3 % page breaks will occur for depths -1, 0, and 1
\newtoks\gttitle % title of current major group
\newskip\intersecskip \intersecskip=12pt minus 3pt % space between sections
\let\yskip=\smallskip
\def\?{\mathrel?}
\def\note#1#2.{\Y\noindent\hangindent2em%
\baselineskip10pt\eightrm#1~\ifacro{\pdfnote#2.}\else#2\fi.\par}}

\newtoks\toksA \newtoks\toksB \newtoks\toksC \newtoks\toksD
\newtoks\toksE \newtoks\toksF \newtoks\usersanitizer
\newcount\countA \countA=0 \newcount\countB \countB=0
\newcount\countC \countC=0
\newif\iftokprocessed \newif\ifTnum \newif\ifinstr
{\def\{\global\let\spacechar= }\ }

\ifacro % The following are pdf macros
\def\thewidth{\the\wd0 \space}
\def\theheight{\the\ht\strutbox\space}

```

```

\def\thedepth{\the\dp\strutbox\space}
\ifpdf
\ifpdf
\ifx\pdfannotlink\undefined\let\pdfannotlink\pdfstartlink\fi% for pdfTeX 0.14
\def\pdflink#1#2{\hbox{\pdfannotlink height\ht\strutbox depth\dp\strutbox
attr{/Border [0 0 0]} goto num #1 \Blue #1\Black\pdfendlink}} % changed 3.69
\else\def\pdflink#1#2{\setbox0=\hbox{\special{pdf: bc [ \pdflinkcolor ]}{#1}%
\special{pdf: ec}}\special{pdf: ann width \thewidth height \theheight
depth \thedepth << /Type /Annot /Subtype /Link
/Border [0 0 0] /A << /S /GoTo /D (#2) >> >>}\box0\relax}\fi
\def\pdfnote#1.{\setbox0=\hbox{\toksA={#1.}\toksB={}\maketoks}\the\toksA}
\def\firstsecno#1.{\setbox0=\hbox{\toksA={#1.}\toksB={}}%
\def\makenote{\addtokens\toksB{\the\toksC}\def\makenote{\toksD={}\toksC={}\let\space\empty}\makenote}\maketoks}
\def\addtokens#1#2{\edef\addtoks{\noexpand#1={\the#1#2}}\addtoks}
\def\poptoks#1#2|ENDTOKS|{\let\first=#1\toksD={#1}%
\ifcat\noexpand\first0\countB=#1\else\countB=0\fi\toksA={#2}}
\def\maketoks{\expandafter\poptoks\the\toksA|ENDTOKS|}%
\ifnum\countB>'9 \countB=0 \fi
\ifnum\countB<'0
\ifnum0=\countC\else\makenote\fi
\ifx\first.\let\next=\maketoksdone\else
\let\next=\maketoks
\addtokens\toksB{\the\toksD}
\ifx\first,\addtokens\toksB{\space}\fi
\fi
\else \addtokens\toksC{\the\toksD}\global\countC=1\let\next=\maketoks
\fi
\next
}
\def\makenote{\addtokens\toksB
{\noexpand\pdflink{\the\toksC}{\romannumeral\the\toksC}}\toksC={}\global\countC=0}
\def\maketoksdone{\edef\st{\global\noexpand\toksA={\the\toksB}}\st}
\def\pdfURL#1#2{\ifpdf\pdfannotlink height\ht\strutbox depth\dp\strutbox
attr{/Border [0 0 0]} user { /Type /Action /Subtype /Link /A
<< /S /URI /URI (#2) >>}\Blue #1\Black \pdfendlink % changed in 3.69
\else \ifpdf{\setbox0=\hbox{\special{pdf: bc [ \pdflinkcolor ]}{#1}%
\special{pdf: ec}}\special{pdf: ann width \thewidth\space height \theheight
\space depth \thedepth\space << /Border [0 0 0]
/Type /Action /Subtype /Link /A << /S /URI /URI (#2) >> >>}\box0\relax}%
\else #1 ({\tt#2})\fi\fi}
{\catcode'\~ =12 \gdef\TILDE/{~} % ~ in a URL
{\catcode'\_ =12 \gdef\UNDER/{_} % _ in a URL
\def\sanitizecommand#1#2{\addtokens\usersanitizer
{\noexpand\dosanitizecommand\noexpand#1{#2}}}%
\def\dosanitizecommand#1#2{\ifx\nxt#1\addF{#2}\fi}

\catcode'\[ =1 \catcode'\] =2 \catcode'\{ =12 \catcode'\} =12
\def\lbchar[{} \def\rbchar[]]
\catcode'\[ =12 \catcode'\] =12 \catcode'\{ =1 \catcode'\} =2
\catcode'\~ =12 \def\tildechar{~} \catcode'\_ =13
\catcode'\| =0 \catcode'\| =12 \def\bschar{\|} \catcode'\| =0 \catcode'\| =12
\def\makeoutlinetoks{\Tnumfalse\afterassignment\makeolproctok\let\nxt=}
\def\makeolnexttok{\afterassignment\makeolproctok\let\nxt=}
\def\makeolgobbletok{\afterassignment\makeolnexttok\let\nxt=}
\def\addF#1{\addtokens\toksF{#1}\toksFprocessedtrue}
% now comes a routine to "sanitize" section names, for pdf outlines
\def\makeolproctok{\toksFprocessedfalse
\let\nxt\makeolnexttok % default
\ifx\nxt\outlinedone\let\nxt\outlinedone
\else\ifx\nxt\else\ifx\nxt\Tnumfalse\instrfalse % skip braces
\else\ifx$\nxt % or a $ sign
\else\ifx~\nxt \addF~\else\ifx\_ \nxt \addF\_ sanitize ~ and _
\else\ifx\nxt\spacechar \addF\space
\else\if\noexpand\nxt\relax % we have a control sequence; is it one we know?
\ifx\nxt~\addF\space
\else\ifx\nxt\onespace\addF\space
\else\the\usersanitizer
\iftokprocessed\else\makeolproctokctli
\iftokprocessed\else\makeolproctokctlii
\iftokprocessed\else\makeolproctokctliii % if not recognised, skip it
\fi\fi\fi\fi\fi

```

```

\else % we don't have a control sequence, it's an ordinary char
\ifx\nxt\addF\string\}% quote chars special to PDF with backslash
\else\ifx\nxt\addF\string\}%\else\ifx\nxt\addF\string\}%
\else\ifx\nxt\addF\string\}%\else\ifx\nxt\addF\string\}%
\else\expandafter\makeolproctokchar\meaning\nxt
\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi
\next
}
\def\makeolproctokchar#1 #2 #3{\addF{#3}}
\def\makeolproctokctlii{%
\ifx\nxt\CEE\addF{C}\let\next\makeolgobbletok % \CEE/
\else\ifx\nxt\UNIX\addF{UNIX}\let\next\makeolgobbletok % \UNIX/
\else\ifx\nxt\TeX\addF{TeX}\let\next\makeolgobbletok % \TeX/
\else\ifx\nxt\LaTeX\addF{LaTeX}\let\next\makeolgobbletok % \LaTeX/
\else\ifx\nxt\CPLUSPLUS\addF{C++}\let\next\makeolgobbletok % \CPLUSPLUS/
\else\ifx\nxt\Cee\addF{C}%
\else\ifx\nxt\PB\let\next\makeolgobbletok\tokprocessedtrue % \PB{...}
\else\ifx\nxt\.\tokprocessedtrue\instrtrue % \.{...}
% skip \
\else\ifx\nxt\ifinstr\addF{\bschar\bschar}\else\tokprocessedtrue\fi
\else\ifx\nxt&\ifinstr\addF&\else\tokprocessedtrue\fi
\else\ifx\nxt\ifTnum\addF{0}\else\addF{tildechar}\fi % 077->\T{~77}
\else\ifx\nxt\ifTnum\addF{E}\else\addF_{\fi % 0.1E5->\T{0.1\_5}
\else\ifx\nxt\ifTnum\addF{0x}\else\addF^{\fi % 0x77 -> \T{^77}
\else\ifx\nxt\ifTnum\tokprocessedtrue\else\addF$\fi % \T{77}$L}
\else\ifx\nxt\{\addF\lchar\else\ifx\nxt\}\addF\rchar
\else\ifx\nxt\ \addFspace\else\ifx\nxt\#\addF{\string\#}%
\else\ifx\nxt\PP\addF{++}\else\ifx\nxt\MM\addF{--}%
\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi
}
\def\makeolproctokctlii{%
\ifx\nxt\MG\addF{->}\else\ifx\nxt\GG\addF{>>}%
\else\ifx\nxt\LL\addF{<<}\else\ifx\nxt\NULL\addF{NULL}%
\else\ifx\nxt\AND\addF&\else\ifx\nxt\OR\addF|}%
\else\ifx\nxt\XOR\addF^\else\ifx\nxt\CM\addF{tildechar}
\else\ifx\nxt\MOD\addF{\string\}%\else\ifx\nxt\DC\addF{::}%
\else\ifx\nxt\PA\addF{.}%\else\ifx\nxt\MGA\addF{->*}%
\else\ifx\nxt>this\addF{this}\else\ifx\nxt\?\addF?%
\else\ifx\nxt\E\addF{==}\else\ifx\nxt\G\addF{>=}%
\else\ifx\nxt\I\addF{!=}\else\ifx\nxt\K\addF{=}%
\else\ifx\nxt\l\addF{l}\else\ifx\nxt\L\addF{L}%
\else\ifx\nxt\o\addF{o}\else\ifx\nxt\O\addF{O}%
\else\ifx\nxt\R\addF{R}%
\else\ifx\nxt\T\let\numtrue\let\next\makeolgobbletok
\tokprocessedtrue % \T{number}
\else\ifx\nxt\AM\addF&\else\ifx\nxt\%\addF{\string\}%
\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi
\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi
}
\def\makeolproctokctliiii{%
\ifx\nxt\V\addF{||}\else\ifx\nxt\W\addF{&&}\else\ifx\nxt\Z\addF{<=}%
\else\ifx\nxt*\addF*\else\ifx\nxt\Xand\addF{\space and\space}%
\else\ifx\nxt\Xandx\addF{\space and_eq\space}%
\else\ifx\nxt\Xbitand\addF{\space bitand\space}%
\else\ifx\nxt\Xbitor\addF{\space bitor\space}%
\else\ifx\nxt\Xcompl\addF{\space compl\space}%
\else\ifx\nxt\Xnot\addF{\space not\space}%
\else\ifx\nxt\Xnotx\addF{\space not_eq\space}%
\else\ifx\nxt\Xor\addF{\space or\space}%
\else\ifx\nxt\Xorx\addF{\space or_eq\space}%
\else\ifx\nxt\Xxor\addF{\space xor\space}%
\else\ifx\nxt\Xxorx\addF{\space xor_eq\space}%
\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi
\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi\fi
}
\def\outlinedone{\edef\outlinest{\global\noexpand\toksE={\the\toksF}}%
\outlinest\let\outlinedone=\relax}
\fi % End of pdf macros

\def\lapstar{\rlap{*}}

```

```

\def\stsec{\rightskip=0pt % get out of C mode (cf. \B)
\sfcode'\=1500 \pretolerance 200 \hyphenpenalty 50 \exhyphenpenalty 50
\noindent{\let\*=\lapstar\bf\secstar.\quad}%
\ifpdf\smash{\raise\baselineskip\hbox to0pt{%
% \let\*=\empty\pdfdest num \secstar fith}} % bad space in versions < 3.68
\let\*=\empty\pdfdest num \secstar fith}} % changed in version 3.68
\else\ifpdf\smash{\raise\baselineskip\hbox to0pt{%
\let\*=\empty\special{%
pdf: dest (\romannumeral\secstar) [ @thispage /FitH @ypos ]}}}\fi\fi}
\let\startsection=\stsec
\def\defin#1{\global\advance\ind by 2 \1\&{#1} } % begin 'define' or 'format'
\def\A{\note{See also section}} % xref for doubly defined section name
\def\As{\note{See also sections}} % xref for multiply defined section name
\def\B{\rightskip=0pt plus 100pt minus 10pt % go into C mode
\sfcode'\=3000
\pretolerance 10000
\hyphenpenalty 1000 % so strings can be broken (discretionary \ is inserted)
\exhyphenpenalty 10000
\global\ind=2 \1\ \unskip}
\def\C#1{\5\quad$/\ast\,$\{\cmntfont #1\}$\,\ast/$}
\let\SHC\C % "// short comments" treated like "/* ordinary comments */"
%\def\C#1{\5\quad$\triangleright\,$\{\cmntfont#1\}$\,\triangleleft$}
%\def\SHC#1{\5\quad$\diamond\,$\{\cmntfont#1\}}
\def\D{\defin{\#define}} % macro definition
\let\E=\equiv % equivalence sign
\def\ET{ and } % conjunction between two section numbers
\def\ETs{, and } % conjunction between the last two of several section numbers
\def\F{\defin{format}} % format definition
\let\G=\ge % greater than or equal sign
% \H is long Hungarian umlaut accent
\let\I=\ne % unequal sign
\def\J{\.\@&}} % TANGLE's join operation
\let\K= % assignment operator
%\let\K=\leftarrow % "honest" alternative to standard assignment operator
% \L is Polish letter suppressed-L
\outer\def\M#1{\MN{#1}\ifon\vfil\penalty-100\vfilneg % beginning of section
\vskip\intersecskip\startsection\ignorespaces}
\outer\def\N#1#2#3.{% beginning of starred section
\ifacro{\toksF={}\makeoutlinetoks#3\outlinedone\outlinedone}\fi
\gdepth=#1\gttitle={#3}\MN{#2}%
\ifon\ifnum#1<\secpagedepth \vfil\eject % force page break if depth is small
\else\vfil\penalty-100\vfilneg\vskip\intersecskip\fi\fi
\message{*\secno} % progress report
\def\stripprefix##1>{\}\def\gttitletoks{#3}%
\edef\gttitletoks{\expandafter\stripprefix\meaning\gttitletoks}%
\edef\next{\write\cont{\ZZ{\gttitletoks}{#1}{\secno}}% write to contents file
\{noexpand\the\pageno}{\the\toksE}}\next % \ZZ{title}{depth}{sec}{page}{ss}
\ifpdf\expandafter\edef\csname curr#1\endcsname{\secno}
\ifnum#1>0\countB=#1 \advance\countB by-1
\advancenum{chunk\the\countB.\expnum{curr\the\countB}}\fi\fi
\ifpdf\special{pdf: outline #1 << /Title (\the\toksE) /Dest
[ @thispage /FitH @ypos ] >>}\fi
\ifon\startsection{\bf#3.\quad}\ignorespaces}
\def\MN#1{\par % common code for \M, \N
{\xdef\secstar{#1}\let\*=\empty\xdef\secno{#1}}% remove \* from section name
\ifx\secno\secstar \onmaybe \else\ontrue \fi
\mark{{\tensy x}\secno}{\the\gdepth}{\the\gttitle}}
% each \mark is {section reference or null}{depth plus 1}{group title}
% \O is Scandinavian letter O-with-slash
% \P is paragraph sign
\def\Q{\note{This code is cited in section}} % xref for mention of a section
\def\Qs{\note{This code is cited in sections}} % xref for mentions of a section
\let\R=\not % logical not
% \S is section sign
\def\T#1{\leavevmode % octal, hex or decimal constant
\hbox{$\def{?}{\kern.2em}%
% \def\$$$#1{\egroup_{\rm#1}\bgroup}% suffix to constant % versions < 3.67
\def\$$$#1{\egroup_{\rm#1}\bgroup}% suffix to constant % in version 3.67
\def\_{{\cdot 10^{\aftergroup}}}% power of ten (via dirty trick)
\let\~=\oct \let\~=hex {#1}$}}
\def\U{\note{This code is used in section}} % xref for use of a section

```

```

\def\Us{\note{This code is used in sections}} % xref for uses of a section
\let\V=\lor % logical or
\let\W=\land % logical and
\def\X#1:#2\X{\ifmmode\gdef\XX{\null$\null}\else\gdef\XX{\fi} %$% section name
  \XX$\rangle\,${\let\I=\ne#2\eightrm\kern.5em
    \ifacro{\pdfnote#1.}\else#1\fi}$\,\rangle$\XX}
\def\Y{\par\yskip}
\let\Z=\le
\let\ZZ=\let % now you can \write the control sequence \ZZ
\let\***

\def\Xand{\W} \def\Xandxeq{\MRL{{\AND}{\K}}} \def\Xbitand{\AND}
\def\Xbitor{\OR} \def\Xcompl{\CM} \def\Xnot{\R} \def\Xnotxeq{\I} \def\Xor{\V}
\def\Xorxeq{\MRL{{\OR}{\K}}} \def\Xxor{\XOR} \def\Xxorxeq{\MRL{{\XOR}{\K}}}

%\def\oct{\hbox{\rm\char'23\kern-.2em\it\aftergroup\?\aftergroup}} % WEB style
%\def\hex{\hbox{\rm\char"7D\tt\aftergroup}} % WEB style
\def\oct{\hbox{$\circ$\kern-.1em\it\aftergroup\?\aftergroup}} % CWEB style
\def\hex{\hbox{$\sim$\scriptstyle\#\tt\aftergroup}} % CWEB style
\def\vb#1{\leavevmode\hbox{\kern2pt\vrule\vtop{\vbox{\hrule
  \hbox{\strut\kern2pt\{#1\}\kern2pt}}
  \hrule}\vrule\kern2pt}} % verbatim string

\def\onmaybef{\let\ifon=\maybe} \let\maybe=\iftrue
\newif\ifon \newif\iftitle \newif\ifpagesaved

\newif\ifheader
\def\lheader{\headertrue\mainfont\the\pageno\eightrm\qqquad\grouptitle
  \hfill\title\qqquad\mainfont\topsecno} % top line on left-hand pages
\def\rheader{\headertrue\mainfont\topsecno\eightrm\qqquad\title\hfill
  \grouptitle\qqquad\mainfont\the\pageno} % top line on right-hand pages
\def\grouptitle{\let\i=I\let\j=J\uppercase\expandafter{\expandafter
  \takethree\topmark}}
\def\topsecno{\expandafter\takeone\topmark}
\def\takeone#1#2#3{#1}
\def\taketwo#1#2#3{#2}
\def\takethree#1#2#3{#3}
\def\nullsec{\eightrm\kern-2em} % the \kern-2em cancels \qqquad in headers

\let\page=\pagebody \raggedbottom
% \def\page{\box255 }\normalbottom % faster, but loses plain TeX footnotes
\def\normaloutput#1#2#3{\ifodd\pageno\hoffset=\pageshift\fi
  \shipout\vbox{
    \vbox to\fullpageheight{
      \iftitle\global\titlefalse
      \else\hbox to\pagewidth{\vbox to10pt{\ifodd\pageno #3\else#2\fi}\fi
      \vfill#1}} % parameter #1 is the page itself
    \global\advance\pageno by1}

\gtitle={\CWEB} output} % this running head is reset by starred sections
\mark{\noexpand\nullsec0{\the\gtitle}}
\def\title{\expandafter\uppercase\expandafter{\jobname}}
\def\topofcontents{\centerline{\titlefont\title}\vskip.7in
  \vfill} % this material will start the table of contents page
\def\startpdf{\ifpdf\pdfcatalog{/PageMode /UseOutlines}\else
  \ifpdf{\special{pdf: docview << /PageMode /UseOutlines >>}}\fi}
\def\botofcontents{\vfill
  \centerline{\covernote}} % this material will end the table of contents page
\def\covernote{}
\def\contentspagenumber{0} % default page number for table of contents
\newdimen\pagewidth \pagewidth=6.5in % the width of each page
\newdimen\pageheight \pageheight=8.7in % the height of each page
\newdimen\fullpageheight \fullpageheight=9in % page height including headlines
\newdimen\pageshift \pageshift=\hoffset
% shift righthand pages wrt lefthand ones (changed in version 3.70)
\def\magnify#1{\mag=#1\pagewidth=6.5truein\pageheight=8.7truein
  \fullpageheight=9truein\setpage}
\def\setpage{\hsize\pagewidth\vsize\pageheight} % use after changing page size
\def\contentsfile{\jobname.toc} % file that gets table of contents info
\def\readcontents{\input \contentsfile}
\def\readindex{\input \jobname.idx}

```

```

\def\readsections{\input \jobname.scn}

\newwrite\cont
\output{\setbox0=\page % the first page is garbage
\openout\cont=\contentsfile
\write\cont{\catcode '\noexpand\@=11\relax} % \makeatletter
\global\output{\normaloutput\page\lheader\rheader}}
\setpage
\vbox to \vsize{} % the first \topmark won't be null

\def\ch{\note{The following sections were changed by the change file:}
\let*=\relax}
\newbox\sbox % saved box preceding the index
\newbox\lbox % lefthand column in the index
\def\inx{\par\vskip6pt plus 1fil % we are beginning the index
\def\page{\box255 } \normalbottom
\write\cont{} % ensure that the contents file isn't empty
\write\cont{\catcode '\noexpand\@=12\relax} % \makeatother
\closeout\cont % the contents information has been fully gathered
\output{\ifpagesaved\normaloutput{\box\sbox}\lheader\rheader\fi
\global\setbox\sbox=\page \global\pagesavedtrue}
\pagesavedfalse \eject % eject the page-so-far and predecessors
\setbox\sbox\vbox{\unvbox\sbox} % take it out of its box
\vsize=\pageheight \advance\vsize by -\ht\sbox % the remaining height
\hsize=.5\pagewidth \advance\hsize by -10pt
% column width for the index (20pt between cols)
\parfillskip 0pt plus .6\hsize % try to avoid almost empty lines
\def\lr{L} % this tells whether the left or right column is next
\output{\if L\lr\global\setbox\lbox=\page \gdef\lr{R}
\else\normaloutput{\vbox to\pageheight{\box\sbox\vss
\hbox to\pagewidth{\box\lbox\hfil\page}}}\lheader\rheader
\global\vsize\pageheight\gdef\lr{L}\global\pagesavedfalse\fi}
\message{Index:}
\parskip 0pt plus .5pt
\outer\def I##1, ##2.{\par\hangindent2em\noindent##1:\kern1em
\ifacro\pdfnote##2.\else##2\fi.} % index entry
\def[##1]{\underline{##1}} % underlined index item
\rm \rightskip0pt plus 2.5em \tolerance 10000 \let*=\lapstar
\hyphenpenalty 10000 \parindent0pt
\readindex}
\def\fin{\par\vfill\eject % this is done when we are ending the index
\ifpagesaved\null\vfill\eject\fi % output a null index column
\if L\lr\else\null\vfill\eject\fi % finish the current page
\ifpdf\makebookmarks \fi % added in Version 3.68
\parfillskip 0pt plus 1fil
\def\grouptitle{NAMES OF THE SECTIONS}
\let\topsecno=\nullsec
\message{Section names:}
\output={\normaloutput\page\lheader\rheader}
\setpage
\def\note##1##2.{\quad{\eightrm##1~}\ifacro{\pdfnote##2.}\else{##2}\fi.}}
\def\Q{\note{Cited in section}} % crossref for mention of a section
\def\Qs{\note{Cited in sections}} % crossref for mentions of a section
\def\U{\note{Used in section}} % crossref for use of a section
\def\Us{\note{Used in sections}} % crossref for uses of a section
\def\I{\par\hangindent 2em}\let*==
\ifacro \def\outsecname{Names of the sections} \let\Xpdf\X
% \ifpdf\makebookmarks \pdfdest name {NOS} fitb % in versions < 3.68
\ifpdf\pdfdest name {NOS} fith % changed in version 3.69
\pdfoutline goto name {NOS} count -\secno {\outsecname}
\def\X##1:##2\X{\Xpdf##1:##2\X \firstsecno##1.%
{\toksF={}\makeoutlinetoks##2\outlinedone\outlinedone}}%
\pdfoutline goto num \the\toksA \expandafter{\the\toksE}}
\else\ifpdf
\special{pdf: outline -1 << /Title (\outsecname)
/Dest [ @thispage /FitH @ypos ] >>}
\def\X##1:##2\X{\Xpdf##1:##2\X \firstsecno##1.%
{\toksF={}\makeoutlinetoks##2\outlinedone\outlinedone}}%
\special{pdf: outline 0 << /Title (\the\toksE)
/A << /S /GoTo /D (\romannumeral\the\toksA) >> >>}}
\fi\fi\fi

```



```

\readsections}
\def\makebookmarks{\let\ZZ=\writebookmarkline \readcontents\relax}
\def\expnumber#1{\expandafter\ifx\csname#1\endcsname\relax 0%
  \else \csname#1\endcsname \fi} % Petr Olsak's macros from texinfo.tex
\def\advancenum#1{\countA=\expnumber{#1}\relax \advance\countA by1
  \expandafter\xdef\csname#1\endcsname{\the\countA}}
\def\writebookmarkline#1#2#3#4#5{%
  \let\(\=\let \let\)=\let \let\[\=\let \let\]=\let \let\/\=\let
  \pdfoutline goto num #3 count -\expnumber{chunk#2.#3} {#5}}
\def\conf\par\vfill\eject % finish the section names
% \ifodd\pageno\else\titletrue\null\vfill\eject\fi % for duplex printers
  \rightskip 0pt \hyphenpenalty 50 \tolerance 200
  \setpage \output={\normaloutput\page\lheader\rheader}
  \titletrue % prepare to output the table of contents
  \pageno=\contentspagenumber
  \def\grouptitle{TABLE OF CONTENTS}
  \message{Table of contents:}
  \topofcontents \startpdf
  \line{\hfil Section\hbox to3em{\hss Page}}
  \let\ZZ=\contentsline
  \readcontents\relax % read the contents info
  \botofcontents \end} % print the contents page(s) and terminate
\def\contentsline#1#2#3#4#5{\ifnum#2=0 \smallbreak\fi
  \line{\consetup{#2}#1
    \rm\leaders\hbox to .5em{\.hfil}\hfil
    \ \ifacro\pdflink{#3}{\romannumeral#3}\else#3\fi\hbox to3em{\hss#4}}}}
\def\consetup#1{\ifcase#1 \bf % depth -1 (@**)
  \or % depth 0 (@*)
  \or \hskip2em % depth 1 (@*1)
  \or \hskip4em \or \hskip6em \or \hskip8em \or \hskip10em % depth 2,3,4,5
  \else \hskip12em \fi} % depth 6 or more
\def\noinx{\let\inx=\end} % no indexes or table of contents
\def\nosecs{\let\FIN=\fin \def\fin{\let\parfillskip=\end \FIN}}
  % no index of section names or table of contents
\def\nocon{\let\con=\end} % no table of contents
\def\today{\ifcase\month\or
  January\or February\or March\or April\or May\or June\or
  July\or August\or September\or October\or November\or December\fi
  \space\number\day, \number\year}
\newcount\twodigits
\def\hours{\twodigits=\time \divide\twodigits by 60 \printtwodigits
  \multiply\twodigits by-60 \advance\twodigits by\time :\printtwodigits}
\def\gobbleone{}
\def\printtwodigits{\advance\twodigits100
  \expandafter\gobbleone\number\twodigits
  \advance\twodigits-100 }
\def\TeX{\ifmmode\it\fi
  \leavevmode\hbox{T\kern-.1667em\lower.424ex\hbox{E}\hskip-.125em X}}
\def\,\{\relax\ifmmode\mskip\thinmuskip\else\thinspace\fi}
\def\datethis{\def\startsection{\leftline{\sc\today\ at \hours}\bigskip
  \let\startsection=\stsec\stsec}}
  % say 'datethis' in limbo, to get your listing timestamped before section 1
%\def\datecontentspage{% versions up to 3.65
% \def\topofcontents{\leftline{\sc\today\ at \hours}\bigskip
% \centerline{\titlefont\title}\vfill}} % timestamps the contents page
\def\datecontentspage{% changed in version 3.66
  \def\botofcontents{\vfill
    \centerline{\covernote}
    \bigskip
    \leftline{\sc\today\ at \hours}} % timestamps the contents page

```

부록 C: CWEB 매크로 쓰는 법

cwebmac의 매크로는 CWEAVE의 출력을 손대지 않고도 여러 가지 모양을 낼 수 있게 해 준다. 이 부록의 목적은 그 가운데 몇 가지 가능성을 설명하는 것이다.

1. PLAIN 형식의 표준 글꼴에 더해 네 가지 글꼴이 선언되어 있다. ‘{\mc UNIX}’라고 하면 중간 크기 대문자로 UNIX를 얻고, ‘{\sc STUFF}’라고 하면 작은 대문자로 STUFF를 얻는다. 그리고 문서의 제목에는 큼직한 글꼴 \titlefont와 \tttitlefont를 고를 수 있는데, \tttitlefont는 타자기꼴이다. 중간 크기 대문자로 UNIX와 C를 가리키는 매크로 \UNIX/와 \CEE/도 있다.

2. TeX 텍스트에서 식별자를 언급할 때는 보통 ‘|identifier|’라고 부른다. 그런데 ‘\{identifier}’라고 해도 된다. 출력은 두 경우가 같아 보이지만, 둘째 방식은 CWEAVE의 C 모드 번역을 거치지 않으므로 identifier를 색인에 넣지 않는다. 둘째 방식에서는 식별자 안의 밑줄 문자마다 앞에 역슬래시를 붙여야 한다.

3. ‘CWEB’을 가리킬 때처럼 타자기꼴을 얻으려면 ‘\.’ 매크로를 쓰면 된다 (이를테면 ‘\.{CWEB}’). 이 매크로의 인자 안에서는 CWEAVE의 색인에 ‘special string characters’로 오른 기호들, 즉 역슬래시나 달러 기호 따위 앞에 역슬래시를 하나 더 넣어야 한다. 여기서 ‘_’는 보이는 빈칸 기호가 된다. 제어열 뒤에 보이지 않는 빈칸을 두려면 ‘_’라고 하면 된다. 문자열이 길면 ‘\’로 갈라 여러 토막으로 나눌 수 있다. 이것은 TeX이 그 자리에서 줄을 바꿔야 할 때 역슬래시를 넣어 준다.

4. 제어열 \pagewidth와 \pageheight, \fullpageheight는 CWEB 파일 첫머리의 limbo에서 다시 정의해 쪽 크기를 바꿀 수 있다. 기본값은

```
\pagewidth=6.5in
\pageheight=8.7in
\fullpageheight=9in
```

이고 이 설명서도 그 값으로 만들었다. \fullpageheight는 \pageheight에 쪽 위의 머리글과 쪽 번호가 들어갈 자리를 더한 것이다. 이 값을 바꾸었다면 바꾼 바로 뒤에 매크로 \setpage를 불러야 한다.

5. \pageshift 매크로는 오른쪽 쪽(즉 홀수 쪽)을 왼쪽(짝수) 쪽에 견주어 오른쪽으로 얼마나 밀지를 정한다. 이 값을 맞추면 양면 인쇄에서 종이의 앞뒤로 쪽 번호가 나란히 놓이게 할 수 있다.

6. \title 매크로는 쪽마다 위쪽에 작은 대문자로 나온다. 따로 정의하지 않으면 job 이름이다.

7. 첫 쪽은 보통 1쪽이 된다. 목차를 15쪽에 두고 16쪽에서 시작하려면 ‘\def\contentspagenumber{15}\pageno=\contentspagenumber \advance\pageno by 1’이라고 적으면 된다.

8. 매크로 \iftitle은 ‘\titletrue’로 정의되어 있으면 머리글 줄을 지운다. 목차를 뺀 보통의 값은 \titlefalse다. 그래서 목차 쪽에는 대개 쪽 번호가 없다.

목차에 손댈 여지를 주는 매크로가 둘 있다. \topofcontents는 목차 정보를 읽기 바로 앞에서 불리고, \botofcontents는 그 바로 뒤에서 불린다. 흔히 쓰는 정의는 이렇다.

```
\def\topofcontents{\null\vfill
\titlefalse % 목차 쪽에도 머리글을 넣는다
\def\rheader{\mainfont The {\tt CWEAVE} processor\hfil}
\centerline{\titlefont The {\tttitlefont CWEAVE} processor}
\vskip 15pt \centerline{(Version 3.64)} \vfill}
```

여기서는 오른쪽 쪽의 머리글인 \rheader를 다시 정의하는 것만으로 목차 쪽 위에 원하는 내용을 올릴 수 있다.

9. 목차 자료는 색인을 TeX으로 처리한 뒤에 읽는 파일에 적힌다. 별표 절마다 한 줄이다. common.toc 파일은 이런 모습일 것이다.

```
\ZZ {Introduction}{0}{1}{28}{-}
\ZZ {The character set}{2}{5}{29}{-}
```

\topofcontents 매크로에서 \ZZ를 다시 정의하면 이 정보를 원하는 어떤 모양으로든 나오게 할 수 있다. (아래 19번도 보라.)

10. CWEB의 출력을 따로 처리할 수 있는 작은 파일로 나뉘어야 하거나 나누는 게 나을 때가 있다. 이를테면 TeX의 목록은 500쪽이 넘는데, 그것만으로도 인쇄 장치나 그 소프트웨어의 용량을 넘기기에 넉넉하다. 아주 큰 작업을 작은 조각으로 자르지 않으면, 어딘가 오류 하나로 모든 과정이 헛되어 처음부터 다시 해야 할 수도 있다.

짜낸 파일을 세 조각으로 안전하게 나누는 방법은 이렇다. 조각을 α , β , γ 라 하고 각 조각이 별표 절로 시작한다고 하자. 모든 매크로는 α 의 첫 limbo 절에서 정의하고, 그 TeX 코드의 사본을 β 와 γ 의 첫머리에도 둔다. 조각을 따로 처리하려면 두 가지를 챙겨야 한다. β 와 γ 의 시작 쪽 번호를 제대로 맞춰야 하고, 세 번의 실행에서 나온 목차 자료를 모아야 한다.

cwebmac 매크로에는 그 일을 거들어 주는 제어열 \contentsfile과 \readcontents가 있다. α 의 limbo 절에 ‘\def\contentsfile{cont1}’을, β 의 limbo 절에 ‘\def\contentsfile{cont2}’를 넣는다. 그러면 TeX이 α 와 β 의 목차 자료를 cont1.tex과 cont2.tex에 적는다. 이제 γ 에서

```
\def\readcontents{\input cont1 \input cont2 \input \contentsfile};
```

이라고 하면 세 조각의 자료를 올바른 순서로 불러온다.

그래도 쪽 번호 문제는 남는다. 한 가지 방법은 β 의 limbo 재료에 다음을 넣는 것이다.

```
\message{Please type the last page number of part 1: }
\read -1 to \temp \pageno=\temp \advance\pageno by 1
```

그러면 TeX이 물을 때 필요한 값을 넣어 주기만 하면 된다. γ 의 첫머리에도 비슷한 구성을 쓴다.

물론 이 방법으로 짜낸 파일을 몇 조각으로든 나눌 수 있다.

11. 색인에 특별한 모양으로 조판된 것을 넣고 싶을 때가 있다. 이를테면 ‘TeX’에 대한 색인 항목을 두고 싶다고 하자. CWEB에서는 (항목이 식별자나 예약어가 아닐 때) 색인 항목을 조판하는 간단한 방법을 둘 준다. ‘@’는 로만체를, ‘@.’는 타자기꼴을 준다. 그런데 이를테면 ‘@~\TeX@’라고 해서 ‘TeX’을 로만체로 조판하려 하면 역슬래시가 걸림돌이 되어 이 항목이 색인의 T 자리에 놓이지 않는다.

해결책은 ‘@:’ 기능을 써서, 정렬용 열쇠를 지워 버리는 매크로를 이렇게 선언하는 것이다.

```
\def\9#1{}
```

이제 CWEB 파일에 이를테면 ‘@:TeX}{\TeX@>’라고 적으면 된다. CWEB에서는 정렬용 열쇠에 따라 이것을 가나다순 자리에 놓고 ‘\9{TeX}{\TeX}’라는 매크로 호출을 만들어 내는데, 그러면 정렬용 열쇠는 찍히지 않는다.

비슷한 수를 절 이름에 감춘 재료를 넣어 원하는 대로 정렬하게 하는 데도 쓸 수 있다. 어떤 이는 이런 재주를 “특별한 세련미”라 하고, 어떤 이는 “꼼수”라 한다.

12. 제어열 \secno는 지금 조판하고 있는 절의 번호로 설정된다.

13. 바뀐 절만 색인과 함께 늘어놓고 싶다면 CWEB 파일에서 첫 절보다 앞선 limbo 절에 ‘\let\maybe=\iffalse’ 명령을 두라. 이것을 변경 파일의 첫 변경으로 삼는 것이 관례다.

다만 이 기능에는 TeX적인 한계가 있다. \proclaim이나 \+, \newcount처럼 plain TeX이 \outer로 선언한 제어열과 함께 쓸 수는 없다. TeX이 그런 제어열을 조용히 건너뛰기를 마다하기 때문이다. 이 한계를 비켜 가는 한 가지 방법은

```
\fi \let\proclaim\relax \def\proclaim{...} \ifon
```

이라고 적는 것이다. 여기서 \proclaim은 여는 것과 같되 \outer 딱지만 없이 다시 정의된다. (여기서 \fi는 조건부 건너뛰기를 멈추고 \ifon은 그것을 다시 켜다.) 마찬가지로

```
\fi \newcount\n \ifon
```

은 \newcount를 안전하게 쓰는 방법이다. plain TeX에는 \+가 하는 일을 하는, \outer가 아닌 매크로 \tabalign이 이미 있다. 짧은 표기 \+가 좋다면

```
\fi \let\+\tabalign \ifon
```

이라고 하면 된다.

14. 영어가 아닌 언어로 출력을 얻으려면 매크로 \A, \As, \ATH, \ET, \ETs, \Q, \Qs, \U, \Us, \ch, \fin, \con, \today, \datethis, \datecontentspage를 다시 정의하라. CWEB 자체는 고칠 필요가 없다.*

15. 매크로 \noat1, \noinx, \nosecs, \nocon으로 출력의 일부를 골라 지울 수 있다.

* 옮긴이 주: 한국어를 위한 그런 다시 정의는 이 번역본이 쓰고 있는 kotexcweb.tex에 들어 있다. 그 파일은 위의 매크로들과 함께 한글 글꼴과 러닝 헤드, 목차와 절 이름 목록의 제목까지 손보며, luatex으로 조판한다. CWEB 파일의 limbo 첫 줄에 \input kotexcweb이라고 적으면 된다.

16. plain TeX 형식의 모든 악센트와 특수 문자 기호는 *TeXbook* 9장에 적힌 그대로 CWEB 문서에서도 쓸 수 있다. 딱 하나만 다르다. 점 악센트 (보통은 `\.`)는 `\:`로 쳐야 한다.

17. `cwebmac.tex`에 주석으로 막아 둔 줄이 여럿 있는데, 사용자가 골라 쓸 만한 제안들이다. 이를테면 그 가운데 하나는 양면 인쇄기를 쓸 때 빈 쪽을 끼워 준다. 이 설명서의 온전한 판에 실린 부록 D, E, F는 프로그램 목록에서 ‘=’ 자리에 ‘←’를 넣는, 주석으로 막아 둔 선택지를 써서 찍은 것이다. 그 부록을 보면 어느 모양이 마음에 드는지 정하는 데 도움이 될 것이다.

18. Andreas Scherer가 `\pdfURL`이라는 매크로를 보태 주었다. 이것으로 CWEB 파일의 TeX 파트나 C 주석 어디에서나 다음처럼 적을 수 있다.

```
You can send email to \pdfURL{the author}{mailto:andreas.scherer@pobox.com}
or visit \pdfURL{his home page}{http://www.pobox.com/\TILDE/scherer}.
```

PDF 문서에서 첫째 인자는 파란색으로 나오고 눌러서 따라갈 수 있는 글이 된다. Acrobat reader가 제대로 설정되어 있으면 그 링크를 사용자의 브라우저로 넘겨 전자우편 클라이언트나 HTML 뷰어를 연다. 종이 문서에서는 두 인자가 모두 찍힌다 (둘째 것은 괄호 안에 타자기꼴로). 인터넷 주소의 어떤 특수 문자는 조금 성가신 방식으로 다뤄야 CWEB나 TeX이 조판 명령과 헷갈리지 않는다. @에는 @@를, ~에는 \TILDE/를, _에는 \UNDER/를 쓰라.

19. PDF 문서에는 목차의 큰 무리 제목을 모두 늘어놓은 북마크가 들어 있고, @*의 깊이 기능을 썼다면 그 가운데 얼마는 다른 것에 딸린다. 그런 북마크 항목을 “outline”이라고도 한다. 게다가 마지막 무리 제목인 ‘절 이름 목록’을 펼치면 모든 절 이름이 나오므로, Acrobat 사용자는 원하는 절로 손쉽게 옮겨 갈 수 있다.

`cwebmac.tex`의 매크로는 북마크로 나오는 이름을 모두 조심스레 “씻어 낸다”. PDF outline의 제한된 조판 능력에 맞지 않는 특수 문자와 조판 코드를 없애는 것이다. 이를테면 CWEB의 어느 절 이름은 ‘Cases for *case_like*’인데 `cweave.tex`에서는 TeX 코드 ‘Cases for \PB{\{\case_like\}}로 적혀 있다. 씻어 낸 이름은 그저 ‘Cases for *case_like*’다. (.pdf 파일을 만들 때는 .toc 파일에 있는 모든 \ZZ의 다섯째 매개변수가 첫째 매개변수를 씻어 낸 꼴로 설정된다. 위 9번과 아래 20번을 보라.)

대체로 씻어 내기는 TeX 제어열과 중괄호를 없애되 CWEB 자신이 정의한 제어열은 남긴다. 이런 옮김이 웬만하면 잘 들어맞지만, TeX적인 재주를 부려 기본 동작을 눌러 이기고 원하는 어떤 옮김이든 얻을 수 있다. 이를테면

```
\sanitizecommand\foo{bar}
```

뒤로 제어열 `\foo`는 ‘bar’로 씻긴다. 또

```
\def\kluj#1\\{\foo}
```

뒤로 TeX 코드 ‘`\kluj bar\\`’는 ‘foo’로 찍히지만 ‘bar’로 씻긴다. 제어열 `\kluj`와 `\\`가 씻어 내기에서 없어지기 때문이다.

20. 나아가 `\ifheader`를 쓰면 무리 제목을 임의의 씻긴 텍스트로 바꾸면서 러닝 헤드에서의 모양도 함께 바꿀 수 있다. 이를테면 다음 두 줄로 시작하는 CWEB 소스 파일을 생각해 보자.

```
\def\klujj#1\\{\ifheader F00\else foo\fi}
@*Chinese \klujj bar\.
```

이 코드는 ‘Chinese foo’라는 제목의 큰 무리를 여는데, 러닝 헤드는 ‘CHINESE FOO’이고 목차 항목은 ‘Chinese foo’다. 그러나 그에 해당하는 북마크는 ‘Chinese bar’이고, .toc 파일의 항목은 다음과 같다.

```
\ZZ {Chinese \klujj bar\\}{1}{1}{1}{Chinese bar}
```